

SCS1303

Introduction to Software Engineering

Section 5 · Software Design

Prof. K. P. Hewagamage

B.Sc. Computer Science · Year 1, Semester 1

University of Colombo School of Computing

Section 5 — Intended Learning Outcomes

By the end of this section, students should be able to:

- LO1** Explain what software design is and describe its role in the software development lifecycle
- LO2** Describe the design journey from high-level architecture down to detailed component design
- LO3** Explain core design principles — abstraction, modularity, cohesion, coupling, and information hiding
- LO4** Describe common software architectural patterns and explain what problem each one solves
- LO5** Explain the purpose of system modelling and identify when to use six key UML diagram types
- LO6** Describe how design differs in plan-driven and agile contexts
- LO7** Identify tools and AI assistance available for software design — and understand what engineers must still own

Section 5 — Roadmap

Eight sub-topics — from design foundations to modern practice

5.1 What is Software Design?

5.2 The Design Journey — Top Down

5.3 Core Design Principles

5.4 Software Architecture

5.5 System Modelling and UML

5.6 Design in Both Cultures

5.7 Design Specification

5.8 AI and Tools in Software Design

Our Running Example — UCSC Student Portal

We will use one familiar system throughout to illustrate every design concept

A system being built—
you know what it should do, who uses it, and what you expect from it.

Five features :

Course Registration	Students register for courses each semester within their degree programme
Results and Grades	Students view marks; lecturers upload results; registry verifies them
Fee Payment	Students pay tuition and library fees; finance office manages accounts
Timetable	Students and lecturers view weekly class and examination schedules
Facility Booking	Students book laboratories, sports facilities, and meeting rooms online

Section 5.1

What is Software Design?

Bridging from requirements to a working system

What is Software Design?

Definition

Software design is the process of transforming what a system must do (requirements) into a blueprint for how to build it — defining the structure, components, data, and behaviour before coding begins.

Design answers three questions:

What are the parts?

What components, modules, and services make up the system?

How do the parts connect?

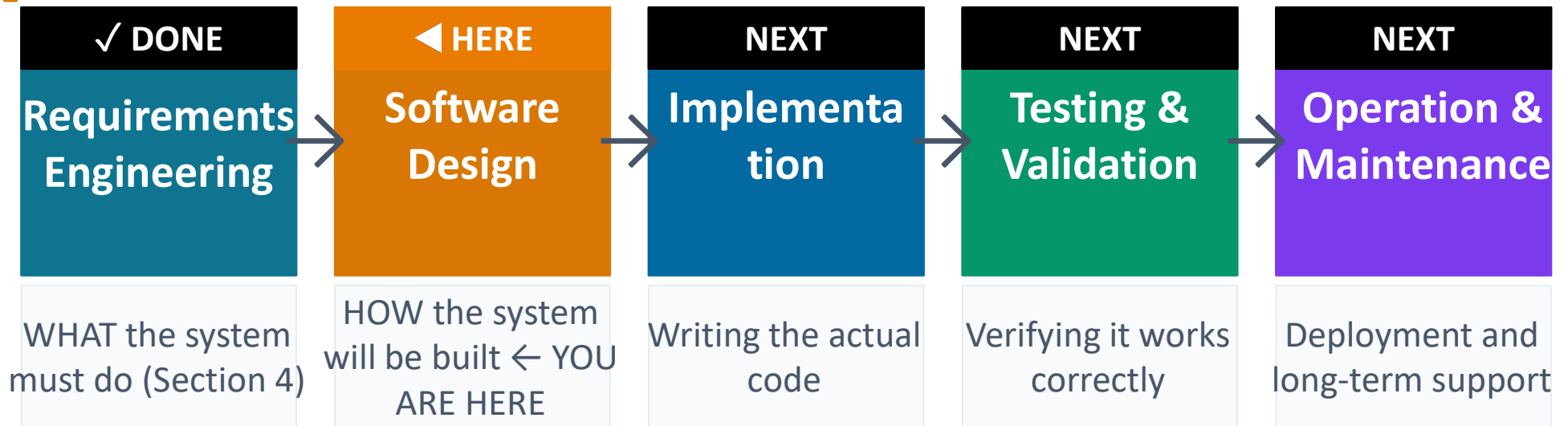
What are the interfaces, data flows, and communication between them?

What does each part do?

What is the responsibility and internal logic of each component?

Design in the Software Lifecycle

Design bridges understanding the problem and building the solution



Design and requirements are not one-way — they inform each other

As you design a system you often discover that **some requirements need clarification**.

In Agile, design and requirements happen concurrently inside each sprint.

In plan-driven projects, design follows requirements — but still feeds back to them.

What Makes a Design Good?

A preview — we explore each of these in depth later

Good design is not just about making something that works. It must also be easy to build, easy to change, and easy to understand.

Correct

Satisfies all requirements — functional and non-functional

Simple

No unnecessary complexity — the simplest design that meets requirements

Modular

Broken into independent pieces — change one without breaking others

Cohesive

Each part does one thing well — not a random mix of unrelated logic

Loosely coupled

Parts depend on each other as little as possible

Documented

Design decisions recorded so others can understand and maintain

Section 5.2

The Design Journey

From requirements down to detailed component design — top-down

Top-Down Design — The Big Idea

Start with the whole system; progressively break it into smaller parts

Top-down design

In top-down design you start at the highest level — the overall system — and progressively refine it into more and more detail, until each part is small enough to be coded directly.

Level 1: Architectural Design

Level 2: Component Design Each major component's responsibilities and internal structure

Level 3: Interface Design

How components communicate — APIs, data formats, screen layouts

Level 4: Data Design

Data structures, database schemas, data that flows between components

Level 5:

Algorithmic Detail

The logic inside individual functions and classes

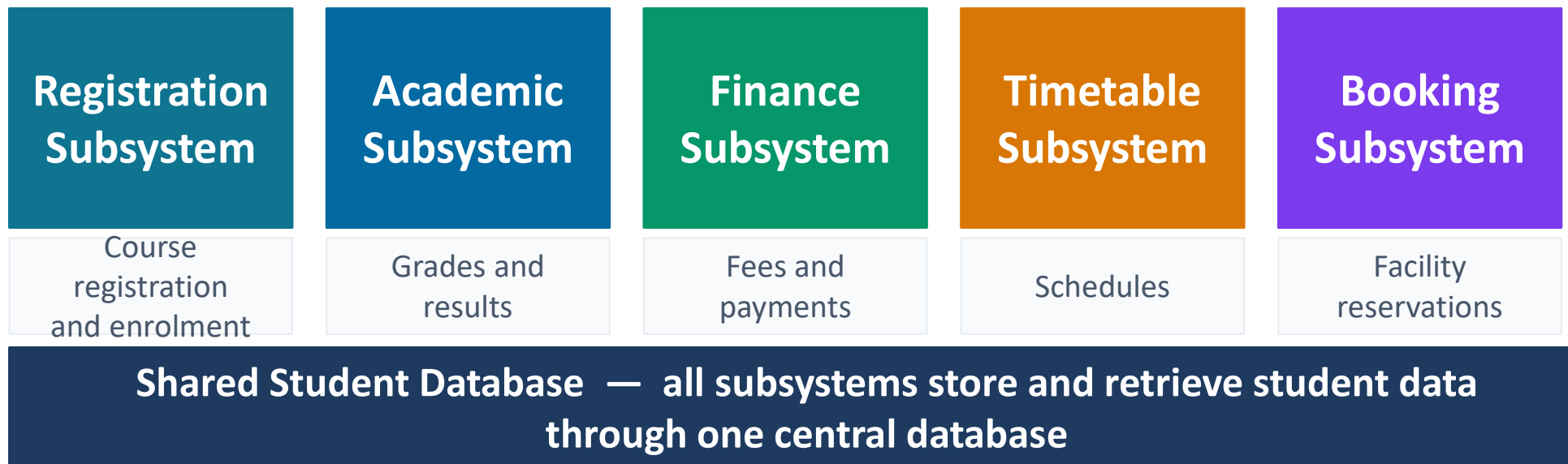
You cannot design the detail until you know the architecture. Architecture is decided first.

Level 1 — Architectural Design

Identifying the major subsystems of the Student Portal

From requirements, we know the portal needs five features.

The first design decision: how do we organise these into major subsystems?

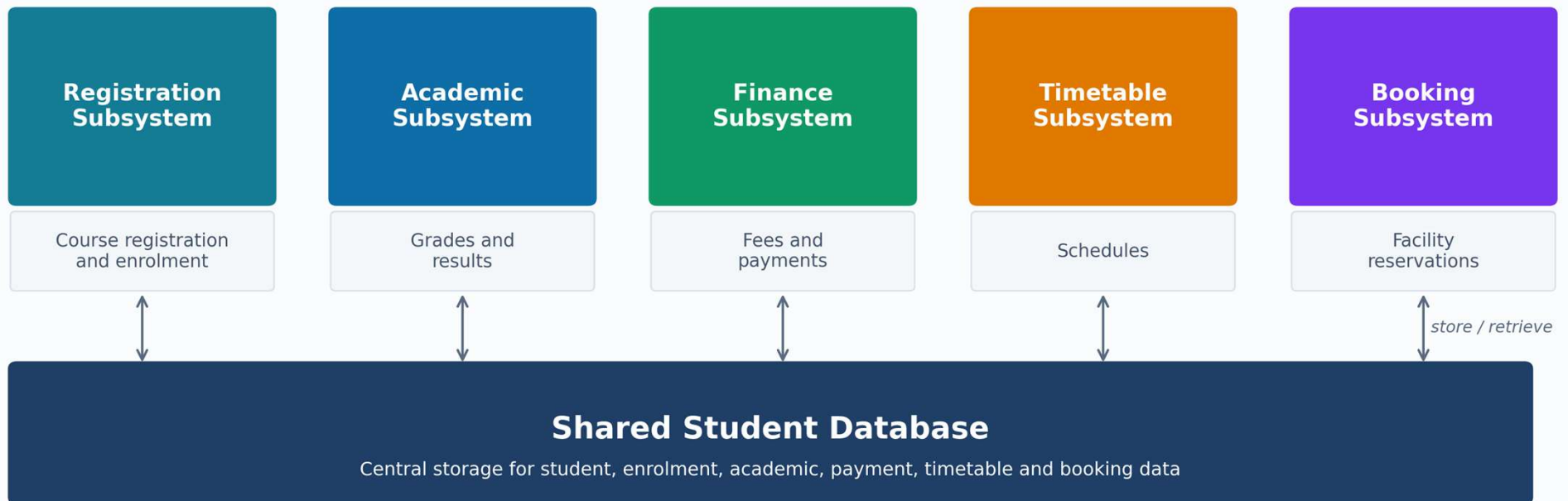


At this level we do NOT yet decide how each subsystem works internally — only that it exists and what it is responsible for.

Running example: UCSC Student Portal

UCSC Student Portal - Level 1 Logical Architecture

Major subsystems and the shared data layer



Authentication, authorization, validation, audit logging and error handling apply across all subsystems.

Level 2 — Component Design

Zooming into one subsystem: the Registration Subsystem

We know what the Registration Subsystem must do.

Now we design how it is structured internally — what components it contains.

Registration Subsystem — internal components

RegistrationController

Receives student requests, validates them, and coordinates the overall flow

CourseAvailabilityChecker

Checks whether a course has open spaces and whether prerequisites are met

TimetableConflictChecker

Checks whether the selected course clashes with already registered courses

EnrolmentManager

Records the enrolment and triggers the confirmation notification

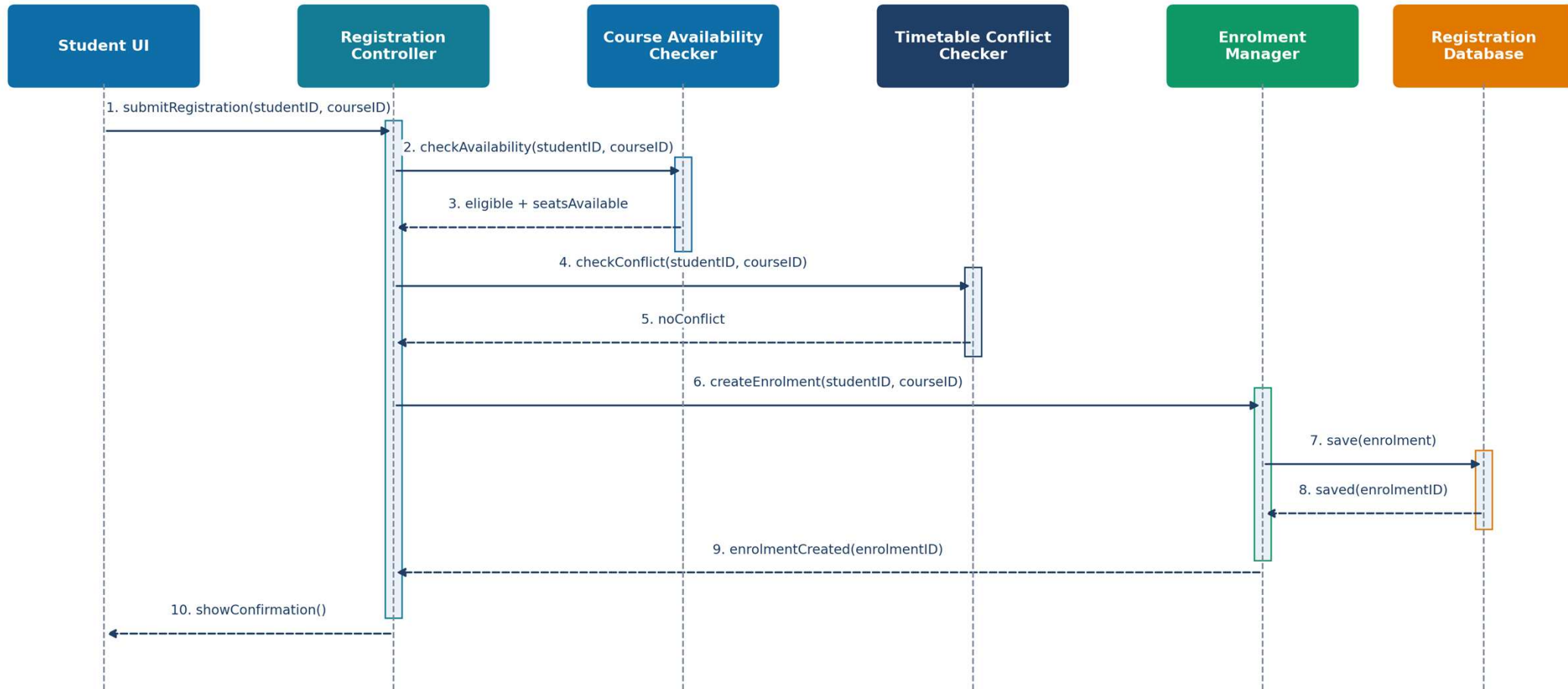
RegistrationDatabase

Stores all enrolment records persistently

Running example: UCSC Student Portal

Runtime Collaboration - Register for a Course

Numbered requests show how the Level 2 components cooperate



Failure paths return a validation, availability, prerequisite or timetable-conflict error before an enrolment is saved.

Level 3 — Interface Design

Defining how users and software components communicate with the system

Interface design is not only API design. It has two important sides:

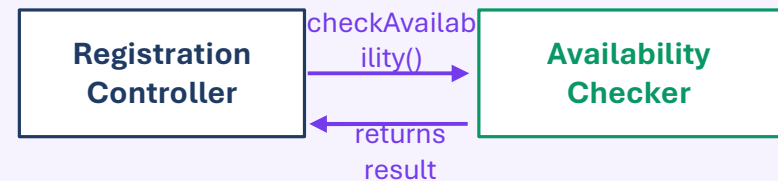
3A. User Interface Design



External user ↔ system

Defines screens, forms, buttons, validation messages, and navigation.

3B. System Interface Design



Component ↔ component

Defines exact operations, inputs, outputs, errors, and rules between modules.

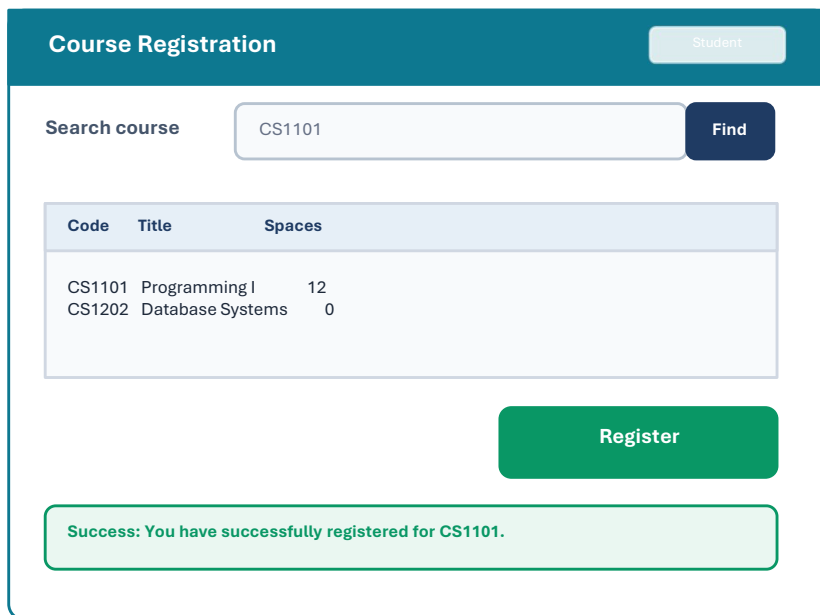
Main point: Level 3 defines the contracts for communication — either with people or between software components.

Running example: UCSC Student Portal

Level 3A — User Interface Design

Designing the screens through which external users interact with the Student Portal

Example screen: Course Registration



The mockup shows a 'Course Registration' screen. At the top, there's a teal header with the title 'Course Registration' and a 'Student' button. Below the header, there's a search section with the label 'Search course', a text input field containing 'CS1101', and a dark blue 'Find' button. Underneath the search is a table with three columns: 'Code', 'Title', and 'Spaces'. The table contains two rows: 'CS1101 Programming I 12' and 'CS1202 Database Systems 0'. Below the table is a large green 'Register' button. At the bottom, there's a light green success message box that says 'Success: You have successfully registered for CS1101.'

Code	Title	Spaces
CS1101	Programming I	12
CS1202	Database Systems	0

What the UI design document should record

Screen purpose	Why this screen exists and which use case it supports.
Actors	Who can access it, such as Student or Lecturer.
Fields and controls	Inputs, buttons, dropdowns, tables and links.
Navigation flow	Previous screen, next screen and possible paths.
Validation and messages	Required fields, error messages and success messages.

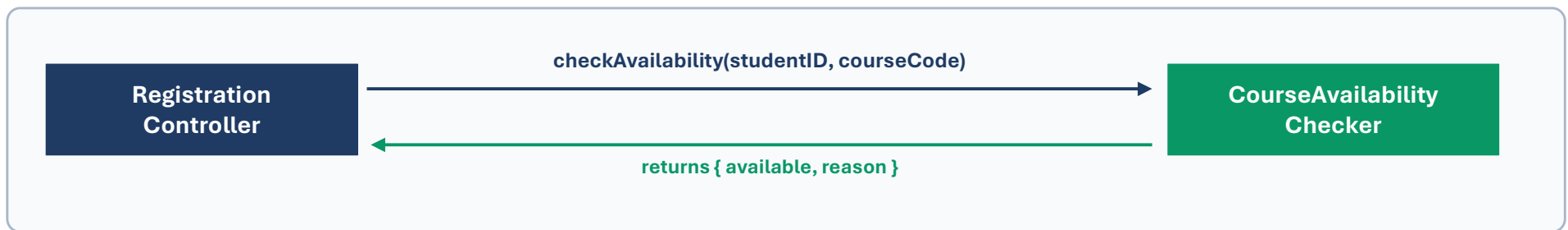
User interface contract: what the user sees, enters, clicks, and receives.

Running example: UCSC Student Portal

Level 3B — System Interface Design

Designing the contract between internal software components

Example: RegistrationController needs CourseAvailabilityChecker to decide whether registration can continue.



Operation contract

Function name
Input parameters
Return type
Who provides it
Who consumes it

Request and response

Request example:
studentID = S1001
courseCode = CS1101

Response example:
available = true

Error contract

Possible failures:

- Course does not exist
- Course is full
- Prerequisite not met
- Timetable conflict

System interface contract: exact inputs, outputs, errors, and rules between components.

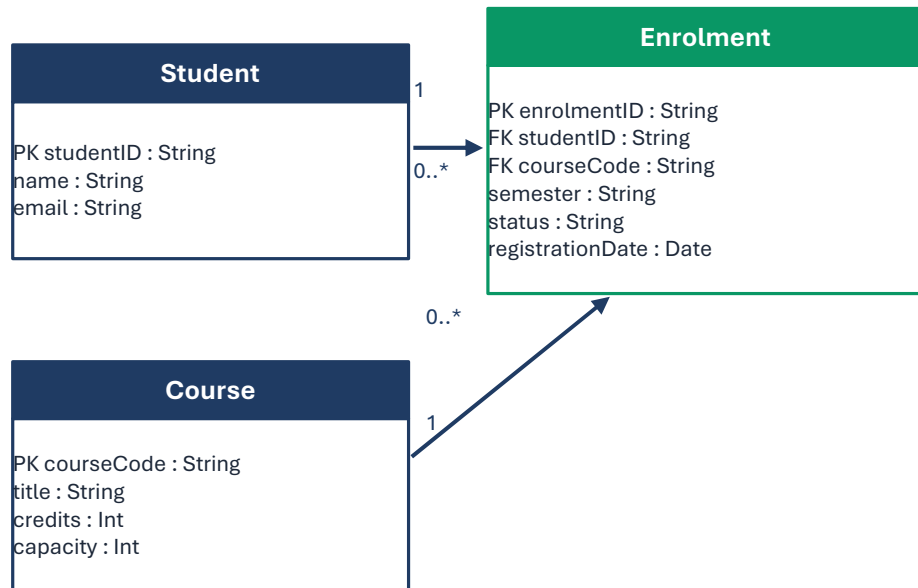
Running example: UCSC Student Portal

Level 4 — Data Design

Designing how portal data is organised, related, protected, and stored

Example data focus: student enrolment records created by the Registration Subsystem.

Logical data model



Database table specification

Table: Enrolment

Column	Type	Rule
enrolmentID	String	Primary key
studentID	String	FK → Student
courseCode	String	FK → Course
semester	String	Required
status	String	Pending / Confirmed / Cancelled
registrationDate	Date	Required

Data design also records:

constraints, indexes, transactions, ownership, security, backup and data retention rules.

Data design contract: what is stored, how data is related, and how integrity is maintained.

Running example: UCSC Student Portal

Levels 3 and 4 — Interface and Data Design

Defining how components talk to each other and how data is organised

Interface Design — the contract between components

The RegistrationController needs to use CourseAvailabilityChecker.

We define the interface — the exact way to call it:

```
checkAvailability( studentID: String, courseCode: String ) → returns: { available: Boolean,  
reason: String }
```

This tells any developer: call this function with these inputs and you will receive this output. No need to know what happens inside.

Data Design — the structure of data in the database

For the Enrolment records, we design the database table:

```
Table: Enrolment → enrolmentID | studentID | courseCode | semester | status | registrationDate
```

Running example: UCSC Student Portal

Section 5.3

Core Design Principles

The ideas that separate good design from bad design

Why Design Principles Matter

Without design principles — a warning story:

A student writes all five portal features in one 3,000-line file. It works at first. Six months later: nobody understands it, a small fee change breaks course registration, and three developers cannot work on it simultaneously. Design principles exist to prevent this.

Five fundamental principles —

Abstraction

Focus on the essential; hide unnecessary detail

Modularity

Divide the system into independent, manageable pieces

Cohesion

Each module does ONE thing — not many unrelated things

Low Coupling

Minimise dependencies between modules

Information Hiding

Hide internal implementation — expose only what is needed

Principle 1: Abstraction

Focus on the essential — hide the unimportant

Definition

Abstraction means representing a system by focusing only on what is relevant for a particular purpose, suppressing everything else. You work with a simplified model of reality.

Abstraction in the Student Portal — four levels:

User level	<i>"I click Register for CS1101" — the student has no idea what happens underneath</i>
Interface level	<i>registerForCourse(studentID, courseCode) — the controller calls this without knowing how it works</i>
Logic level	<i>Check availability → check timetable → check fees → add record → send email</i>
Data level	<i>INSERT INTO Enrolments VALUES (studentID, courseCode, date, status) — the SQL query</i>

Abstraction lets each developer focus on their level without being overwhelmed by the whole system at once.

Principle 2: Modularity

Divide and conquer — break the system into manageable independent pieces

Definition

Modularity means dividing the software into separate, independently developed modules, each with a clearly defined purpose. Modules can be developed, tested, and changed independently.

Without modularity ✗

- One massive portal file — all 5 features together
- Change fee logic → risk breaking registration
- Three developers cannot work simultaneously
- Test one feature requires running everything

With modularity ✓

- 5 separate subsystems — each independent
- Change finance without touching registration
- Three teams build three modules in parallel
- Test each module in isolation — fast and safe

*A modular system is like Lego bricks —
you can add, remove, or replace pieces without rebuilding everything else.*

Principle 3: Cohesion

Every module should do ONE thing — and do it well

Definition

Cohesion measures how closely related the responsibilities within a single module are. A highly cohesive module has one clear purpose — every piece of code inside contributes to that single purpose.

High Cohesion — TimetableConflictChecker

- `checkForConflict( )`
- `getStudentSchedule( )`
- `compareTimeslots( )`
- `returnConflictDetails( )`

All four functions serve ONE purpose: checking timetable conflicts.

This is high cohesion.

Running example: UCSC Student Portal

Low Cohesion — StudentModule (BAD)

- `checkTimetableConflict( )`
- `processFeePayment( )`
- `sendResultsEmail( )`
- `bookSportsFacility( )`

Four completely unrelated responsibilities in one module. A change to fees might affect timetables.

This is low cohesion.

Principle 4: Coupling

How much do modules depend on each other? Less is better.

Definition

Coupling measures the degree of interdependence between modules. Low (loose) coupling means a change in one module is unlikely to force changes in another.

High (tight) coupling creates fragile systems where changes cascade everywhere.

Tight coupling — BAD

- RegistrationController reads Finance database tables directly
- If Finance DB schema changes, Registration code breaks too
- Cannot test Registration without Finance system running
- Developers from both teams must coordinate every change

Loose coupling — GOOD

- RegistrationController calls checkFeeStatus(studentID) only
- Finance team can change internals freely without affecting Registration
- Teams develop and test independently
- Replace the Finance module entirely — Registration never knows

Aim for HIGH cohesion within each module and LOW coupling between modules — these two principles work together.

Principle 5: Information Hiding

What you don't need to know, you shouldn't see

Definition

Information hiding means that the internal implementation details of a module are hidden from the outside world. Other modules only see the interface — the inputs and outputs — not how the work is done internally.

Three benefits for the Student Portal:

Freedom to change

Rewrite the EnrolmentManager to use a different database — as long as the interface stays the same, nothing else changes. No other module knows or cares.

Simpler to use

Developers using EnrolmentManager only need to understand: `saveEnrolment(studentID, courseCode)`. They are not overwhelmed by 300 lines of internal SQL logic.

Prevents misuse

If internal data is not exposed, external code cannot accidentally corrupt it. The database connection is private — only the EnrolmentManager accesses it.

Running example: UCSC Student Portal

The Goal: Separation of Concerns

Every part of the system deals with one concern — and is protected from unrelated change

Why all five principles point to the same goal

Abstraction + Modularity + Cohesion + Low Coupling + Information Hiding all work together to achieve one fundamental goal: **every part of the system deals with its own concern and is insulated from changes in other concerns.**

Applied to the Student Portal:

Fee processing concern

ONLY the Finance Subsystem deals with fee logic. If the government changes the tax rate, only the Finance Subsystem changes. Registration, Grades, and Booking are not affected at all.

Timetable concern

ONLY the Timetable Subsystem manages schedule data. When a new semester is added, only the Timetable Subsystem is updated.

Data storage concern

ONLY the database layer knows HOW data is stored. Switch from one database to another — nothing else in the system needs to change.

User interface concern

ONLY the presentation layer deals with screens. Redesign the portal's look completely — the business logic remains untouched.

Section 5.4

Software Architecture

The big picture — defining structure before the detail

What is Software Architecture?

Definition (IEEE 1471)

Software architecture is the fundamental organisation of a system — its major components, the relationships between them, and the principles guiding their design and evolution.

Architecture ≠ Design ≠ Code

Architecture

The big picture: the structure, patterns, and constraints of the whole system

Design

Decisions inside that structure: component internals, interfaces, data

Code

The realisation of those designs: the actual lines of software

Why Architecture Must Come First

The most important design decisions are the hardest to change later

Architectural decisions affect the entire system. Getting them wrong is very expensive. Getting them right creates a foundation that supports the system for years.

Architecture determines quality

Performance, security, scalability — all determined largely by architecture, not by individual functions

Architecture shapes team structure

Team organisation follows system structure. Define the architecture and you define how teams work.

Architecture is expensive to change

Refactoring a class is cheap. Restructuring the fundamental organisation of a system can cost months.

Architecture enables parallel work

Once component boundaries are defined, many developers can work simultaneously without conflicts.

Architectural Styles/Pattern

- Stylized abstract description of good practice that has been tested in different systems and environments.
- Describe system organization that has been successful in previous systems
- Examples
 - Client-server (2, 3, ...n tire patterns)
 - Layered Architectures
 - Model View Controllers (MVC)

Architectural Patterns

- Patterns are a means of representing, sharing and reusing knowledge.
- An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.
- Patterns should include information about when they are and when they are not useful.
- Patterns may be represented using tabular and graphical descriptions.

Architectural Pattern 1: Layered Architecture

Organise the system into layers — each layer serves the one above it

Concept

Divide the system into horizontal layers. Each layer provides services to the layer above and uses services from the layer below. Layers only communicate with adjacent layers.

Presentation Layer

Student Portal web pages and mobile app screens

Business Logic Layer

Registration rules, grade calculation, fee processing

Data Access Layer

Functions that read from and write to the database

Database Layer

The actual tables: Students, Courses, Enrolments, Fees

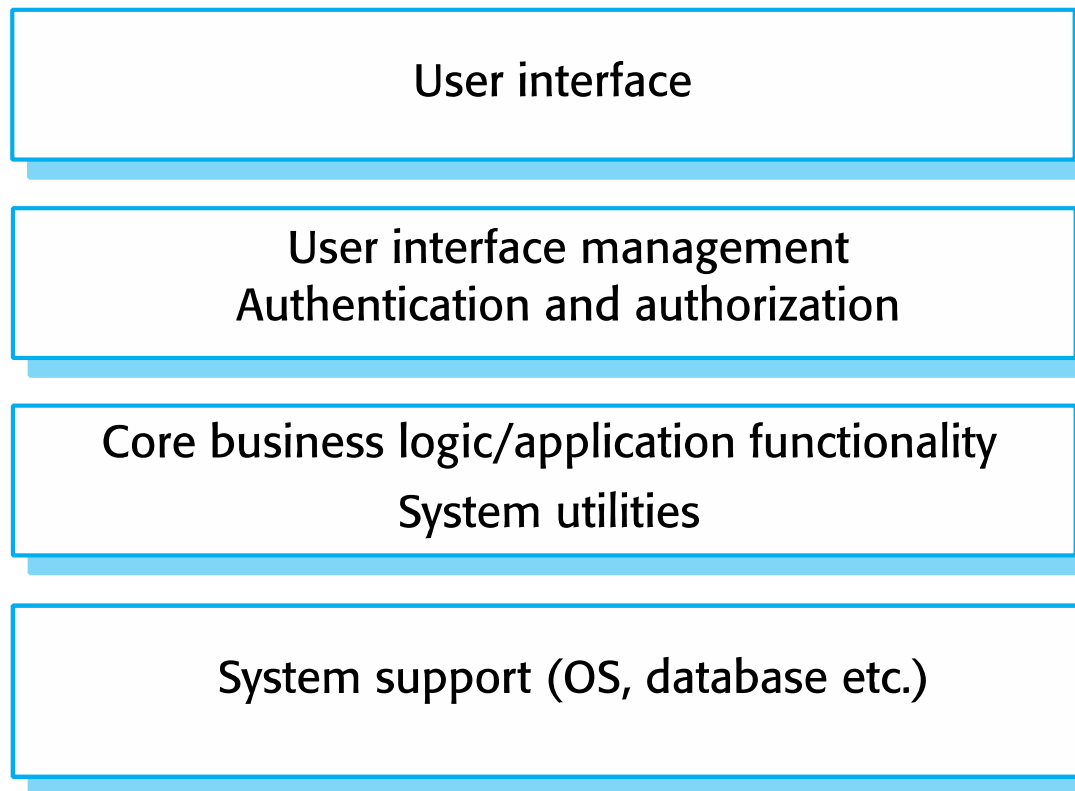
✓ Change the database without touching the UI

✗ Requests travel through every layer — can be slow

Layered Architecture

- Used to model the interfacing of sub-systems.
- Organises the system into a set of layers (or abstract machines) each of which provide a set of services.
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.

A Generic Layered Architecture



The Layered architecture pattern

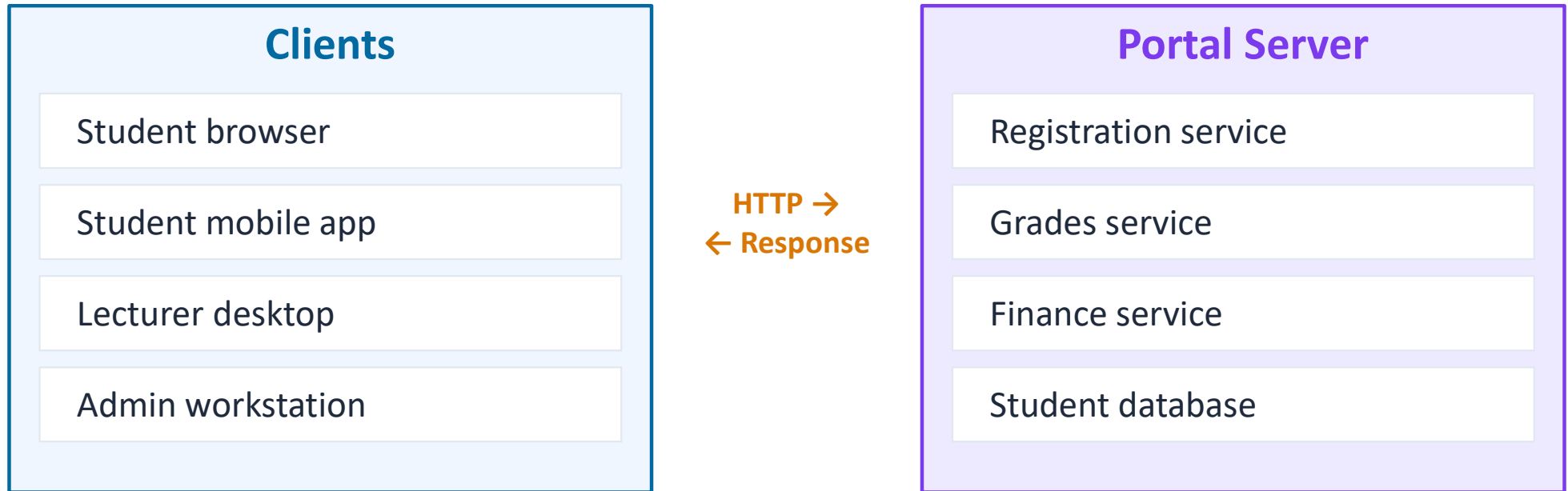
Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system.
Example	A layered model of a system for sharing copyright documents held in different libraries,
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Architectural Pattern 2: Client-Server

Separate what the user sees from where data is processed and stored

Concept

One process (the server) provides services; another process (the client) requests them. Clients and server communicate across a network.

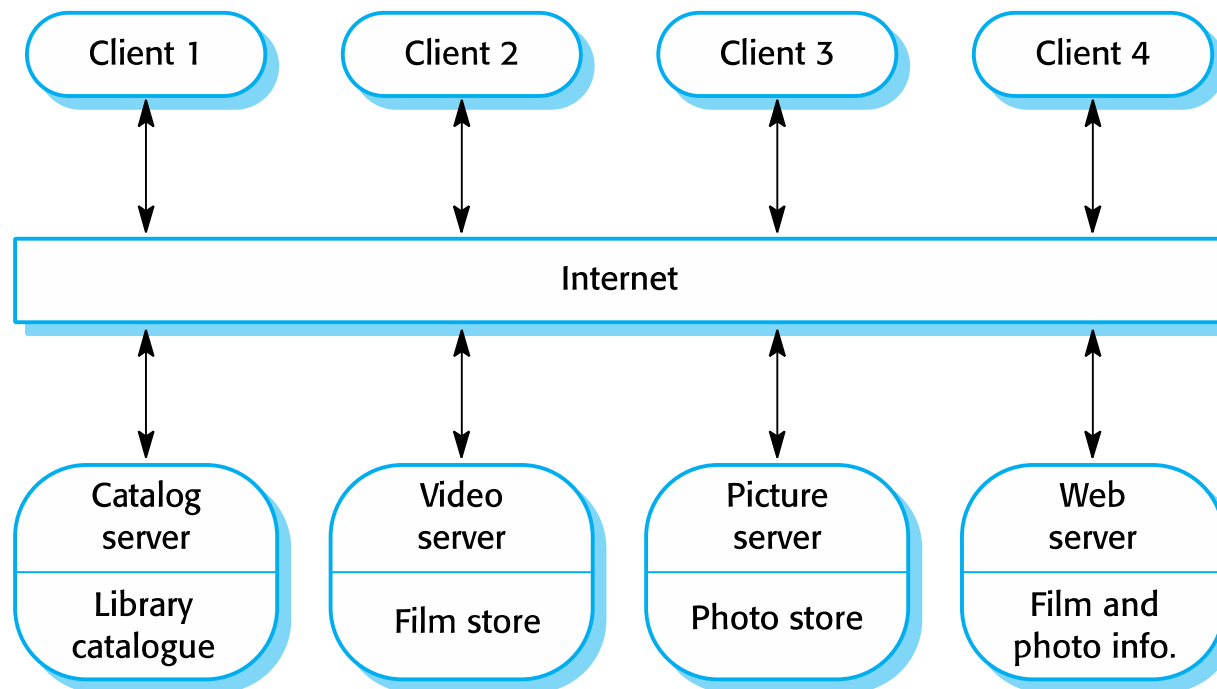


✓ Students access from anywhere · ✓ One server manages all data ·
✗ Server failure affects all users

Client-server architecture

- Distributed system model which shows how data and processing is distributed across a range of components.
 - Can be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.

Example: A client–server architecture for a film library



The Client–server pattern

Name	Client-server
Description	In a client–server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
Example	Different users accessing a website on the server using their individual machines.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial of service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

Architectural Pattern 3: Model-View-Controller (MVC)

Separate the data, the display, and the logic that connects them

Concept

MVC separates a system into: Model (data and business rules), View (what the user sees), and Controller (handles user actions, coordinates model and view).

Model

Manages data and business rules. Holds student records, grades, and fee status. Knows nothing about how data is displayed.

View

The screen the student sees — the course list, results table, payment form. Contains NO business logic.

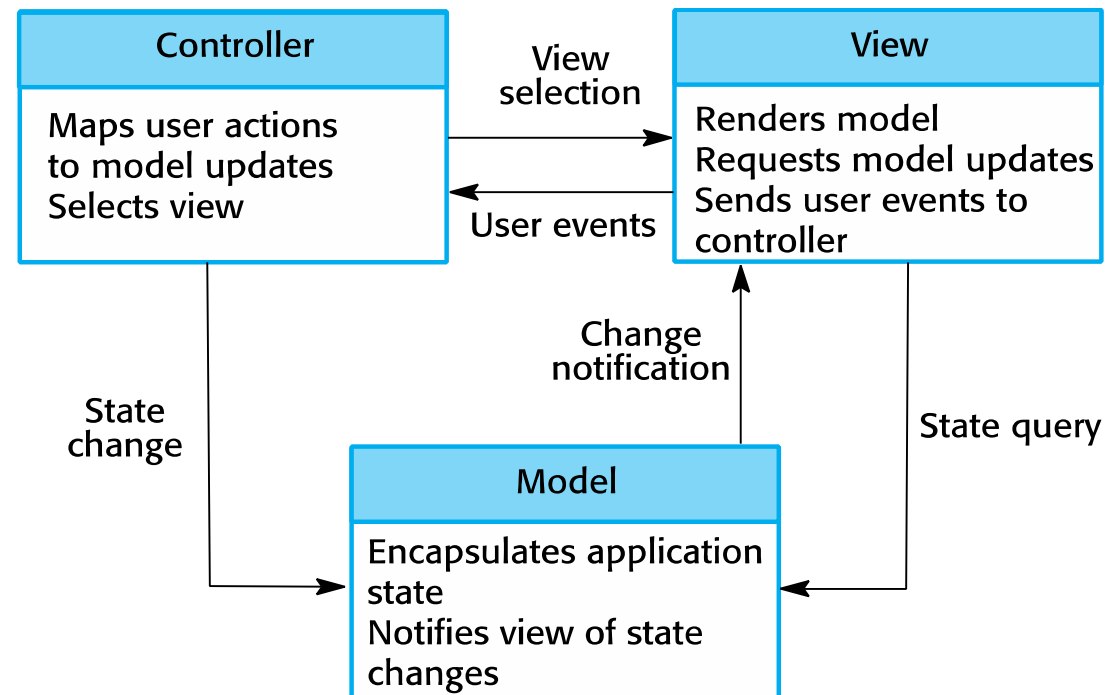
Controller

Receives user actions (button clicks, form submissions). Decides what to update in the Model and what to show in the View.

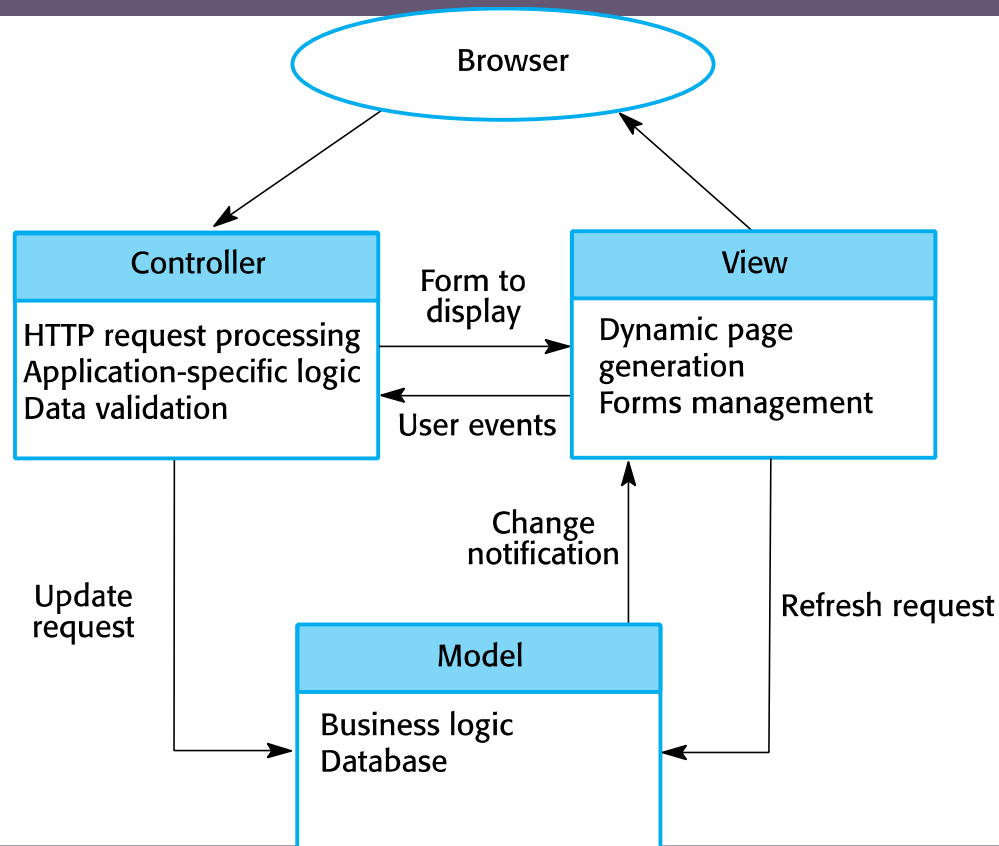
✓ *Change the UI without touching data logic* · ✓ *Same data displayed many ways* ·
Used in Django, Laravel, Spring MVC

Running example: UCSC Student Portal

The organization of the Model-View-Controller



Web application architecture using the MVC pattern



The Model-View-Controller (MVC) pattern

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.
Example	the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways with changes made in one representation shown in all of them.
Disadvantages	Can involve additional code and code complexity when the data model and interactions are simple.

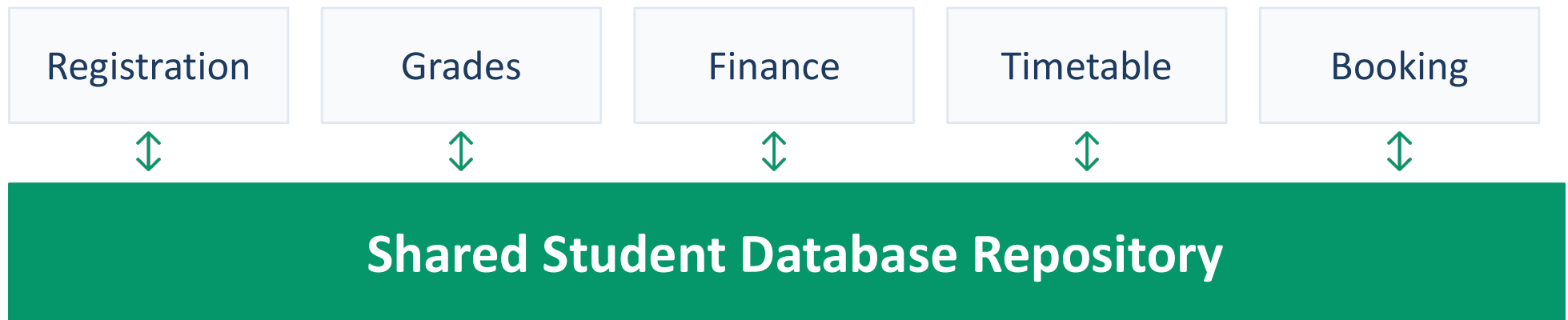
Architectural Pattern 4: Repository

All components share data through one central store — no direct communication

Concept

Components do not interact with each other directly. All shared data is held in a central repository. Components read from and write to this shared store independently.

Student Portal — repository approach:



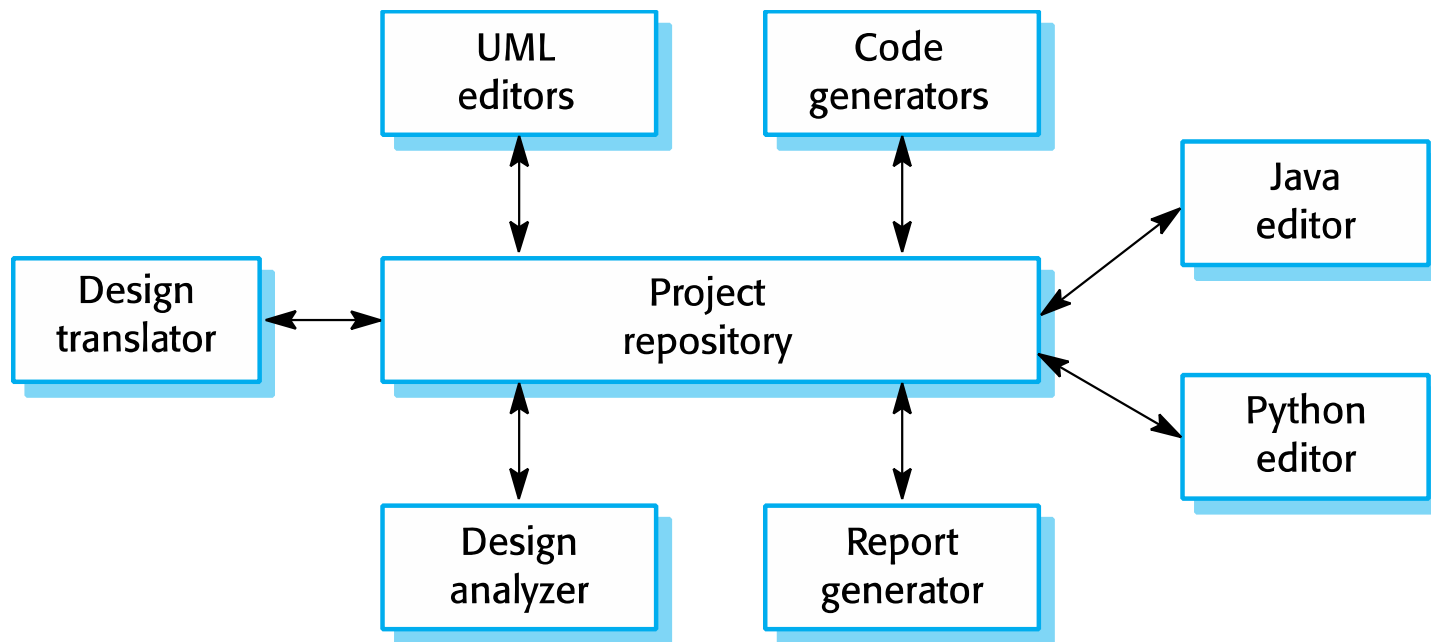
✓ Services independent — add a new one freely
✓ Consistent data management in one place

✗ Repository is a single point of failure
✗ Can become a bottleneck under heavy load

Repository Architecture

- Sub-systems must exchange data. This may be done in two ways:
 - Shared data is held in a central database or repository and may be accessed by all sub-systems;
 - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used this is an efficient data sharing mechanism.

Example: A repository architecture for an IDE



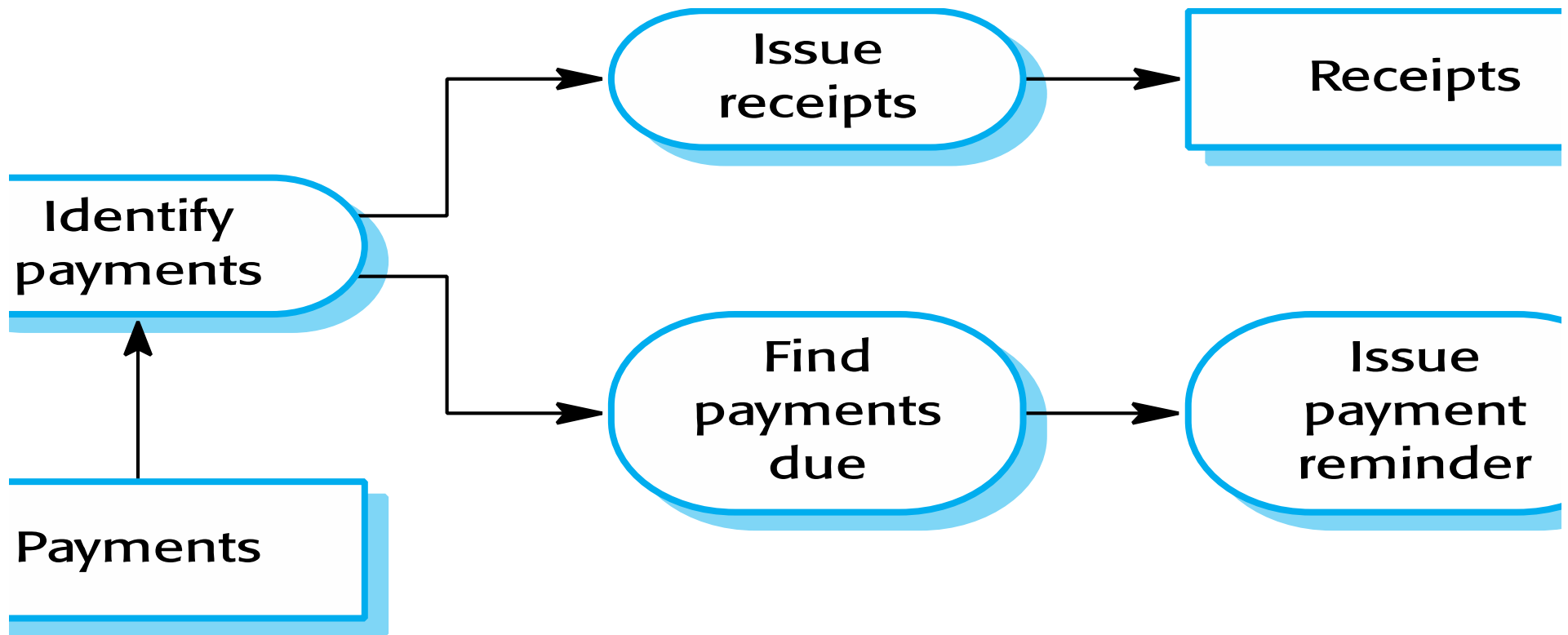
The Repository Pattern

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository.

Pipe and Filter Architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.

An example of the pipe and filter architecture used in a payments system



The pipe and filter pattern

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

Modern Architecture: Microservices

The dominant architecture style in cloud software — an awareness slide

What are microservices?

Each feature of a system runs as its own small, independent service — with its own code, database, and deployment. Services communicate via APIs. Each can be deployed and scaled independently.

Traditional Portal (one application)

- All features deployed together
- Scale everything even if only fees are busy
- One bug can crash the whole portal
- One big team must coordinate everything

Microservices Portal (modern approach)

- Registration, Finance, Grades — separate services
- Scale only the Finance service at fee deadline
- Booking service crashes — grades still work
- Small teams own their own service independently

Microservices are standard in industry — you will encounter this in your career. Deep study comes in advanced courses.

Section 5.5

System Modelling and UML

The language of design — making invisible structure visible

Why We Model Software

A diagram is worth more than a page of explanation

What is system modelling?

System modelling is the process of creating abstract diagrams and models of a system — to help understand, communicate, and analyse its structure and behaviour. A model is a simplified, focused view.

Four reasons for software engineer models:

Communication

A diagram conveys structure faster than paragraphs of text. Developers, clients, and managers can all discuss from the same picture.

Design clarity

Drawing forces you to think the design through. Ambiguities appear before any code is written — when they are cheap to fix.

Documentation

Models serve as a reference for new team members and for maintenance engineers years after the original team has moved on.

Code guidance

Detailed models guide implementation — telling developers what to build before they start writing code.

Four Perspectives on a System

No single diagram shows everything — you need multiple views

A system as complex as the Student Portal must be understood from several angles. Different stakeholders need different perspectives.

External Perspective

"What is outside the system?"

System boundary — what it interacts with

Diagram: Context / Use Case diagram

Interaction Perspective

"How does it interact?"

How users and components communicate

Diagram: Use Case / Sequence diagrams

Structural Perspective

"What is it made of?"

Components, classes, and their relationships

Diagram: Class / Component diagrams

Behavioural Perspective

"How does it behave?"

Responses to events and actions over time

Diagram: Activity / State diagrams

UML — The Standard Modelling Language

One notation, understood worldwide by software engineers

What is UML?

The Unified Modelling Language (UML) is the internationally accepted standard notation for modelling software systems. It provides a set of diagram types covering structural, behavioural, and interaction perspectives. UML 2.5.1 (2017) is the current version, maintained by the Object Management Group (OMG) at omg.org.

UML has 14 diagram types. For this course, you need to know these 6:

Use Case Diagram

Who does what? External view.

Class Diagram

What are the things?
Structure.

Sequence Diagram

Who talks to whom, when?

Activity Diagram

Flow of actions in a process.

Component Diagram

What modules make up the system?

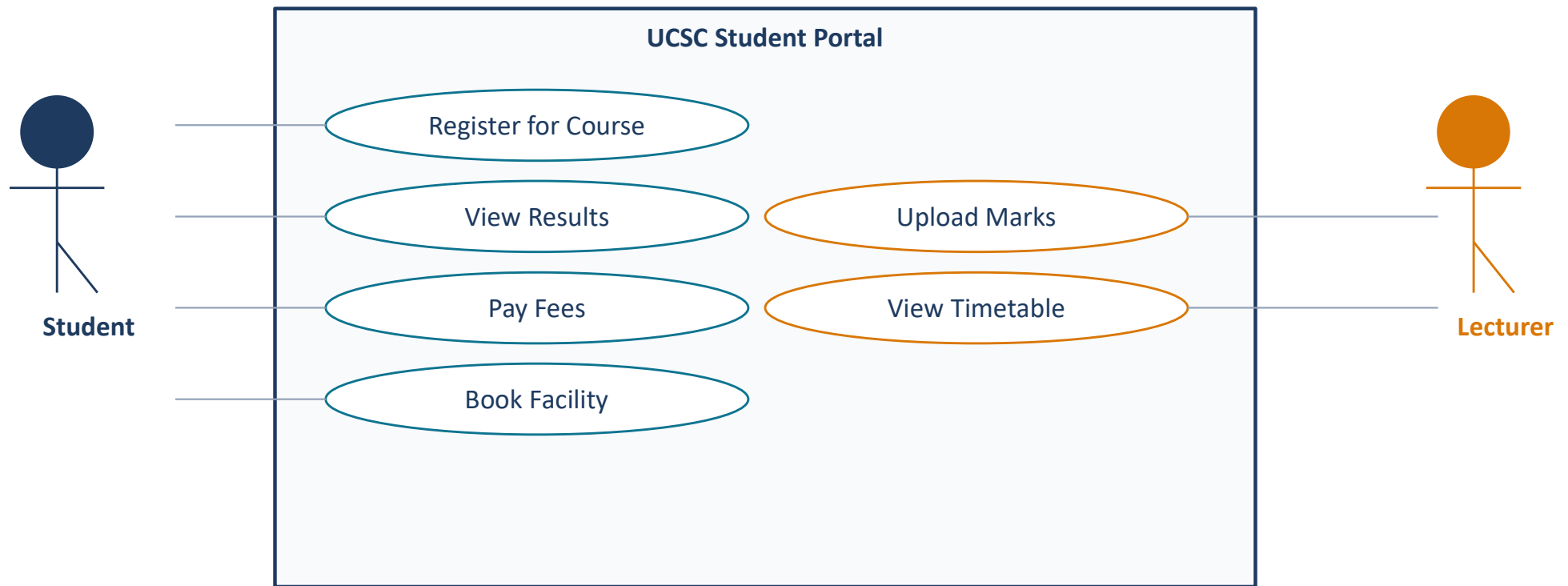
Deployment Diagram

Where does it physically run?

UML 1: Use Case Diagram

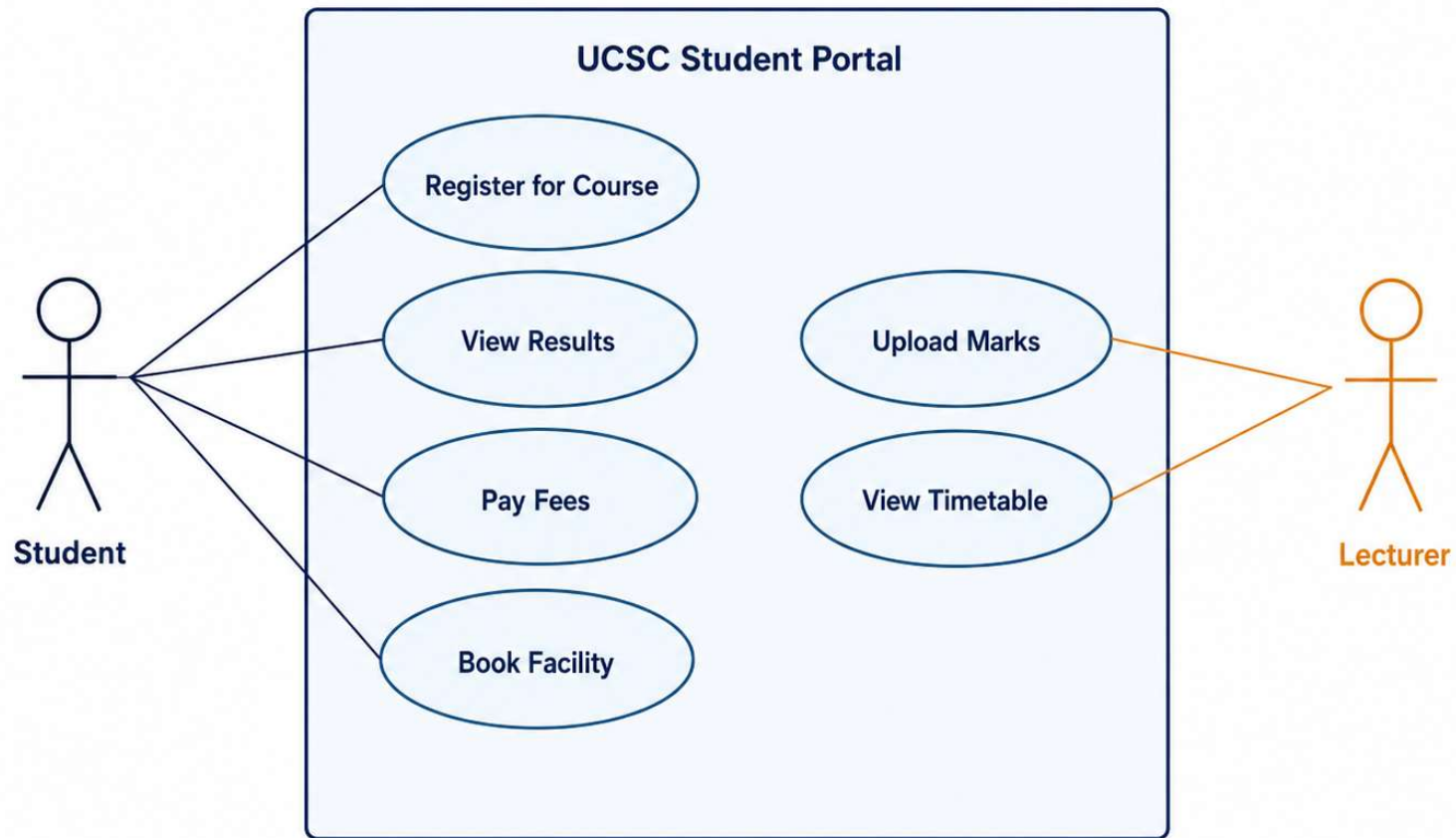
Shows who interacts with the system and what they can do

Answers: Who are the users? What can each user do? What external systems does the portal connect to?



Key notation: *oval = use case · stick figure = actor · rectangle = system boundary · line = actor-use case association*

UML Use Case Diagram – Student Portal

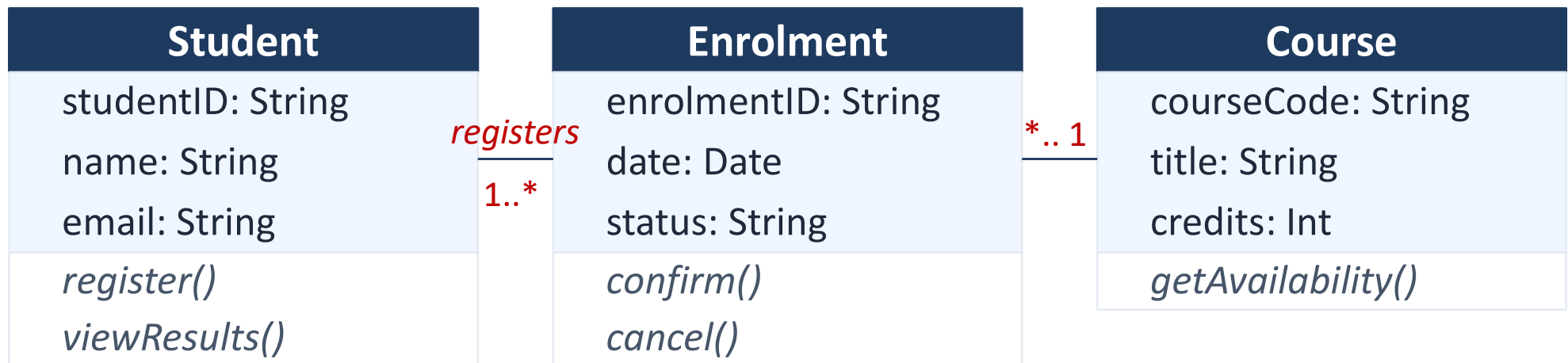


Actor = external user, Oval = use case, Rectangle = system boundary, Line = association

UML 2: Class Diagram

Shows the things in the system, their data, and their relationships

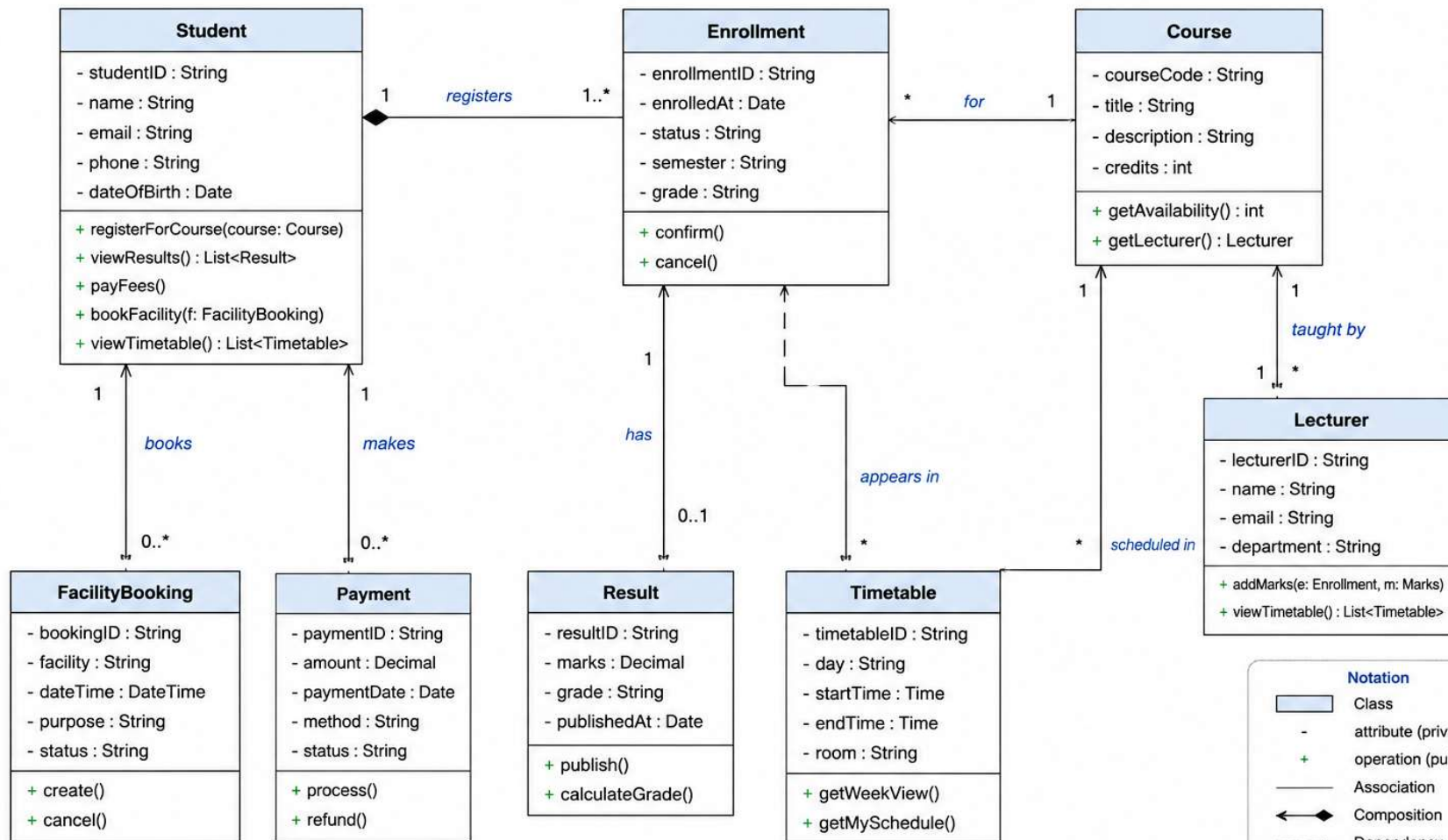
Answers: What are the main entities? What data does each hold? How are they connected to each other?



Reading class diagrams:

- Student 1 — * Enrolment → One student has many enrolments (one per course per semester)
- Enrolment * — 1 Course → Many enrolments reference one course
- Lecturer 1 — * Course → One lecturer teaches multiple courses

UML 2: Class Diagram – UCSC Student Portal

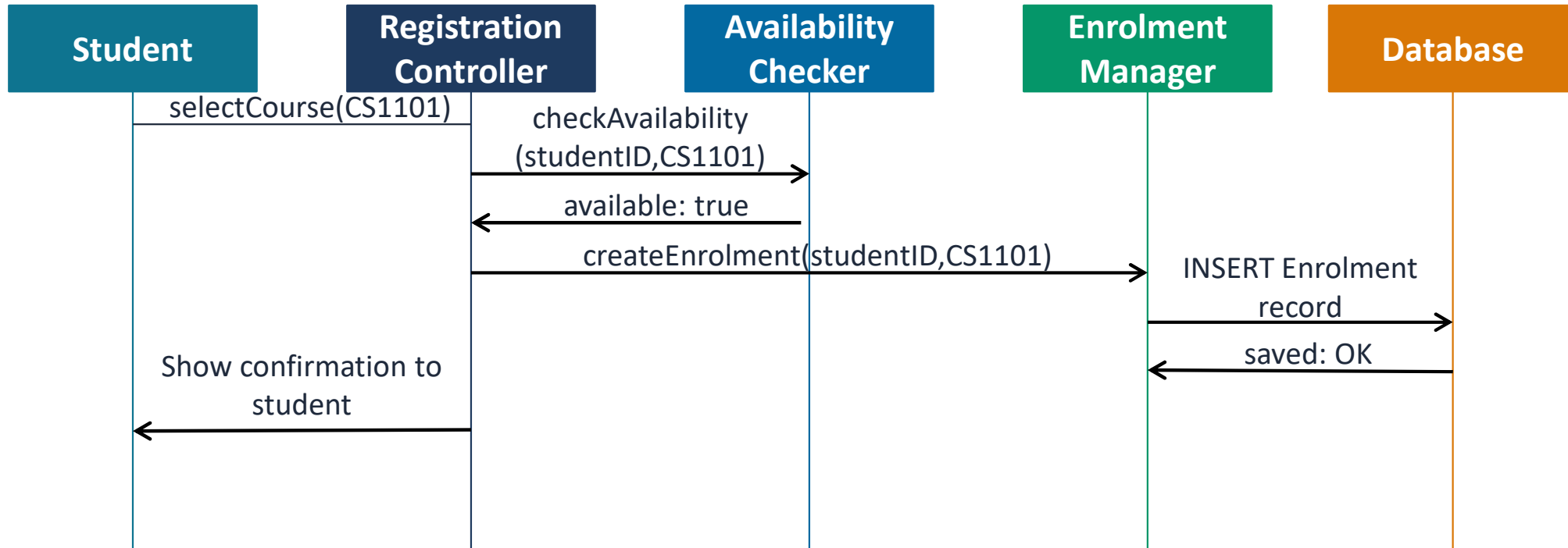


UML 3: Sequence Diagram

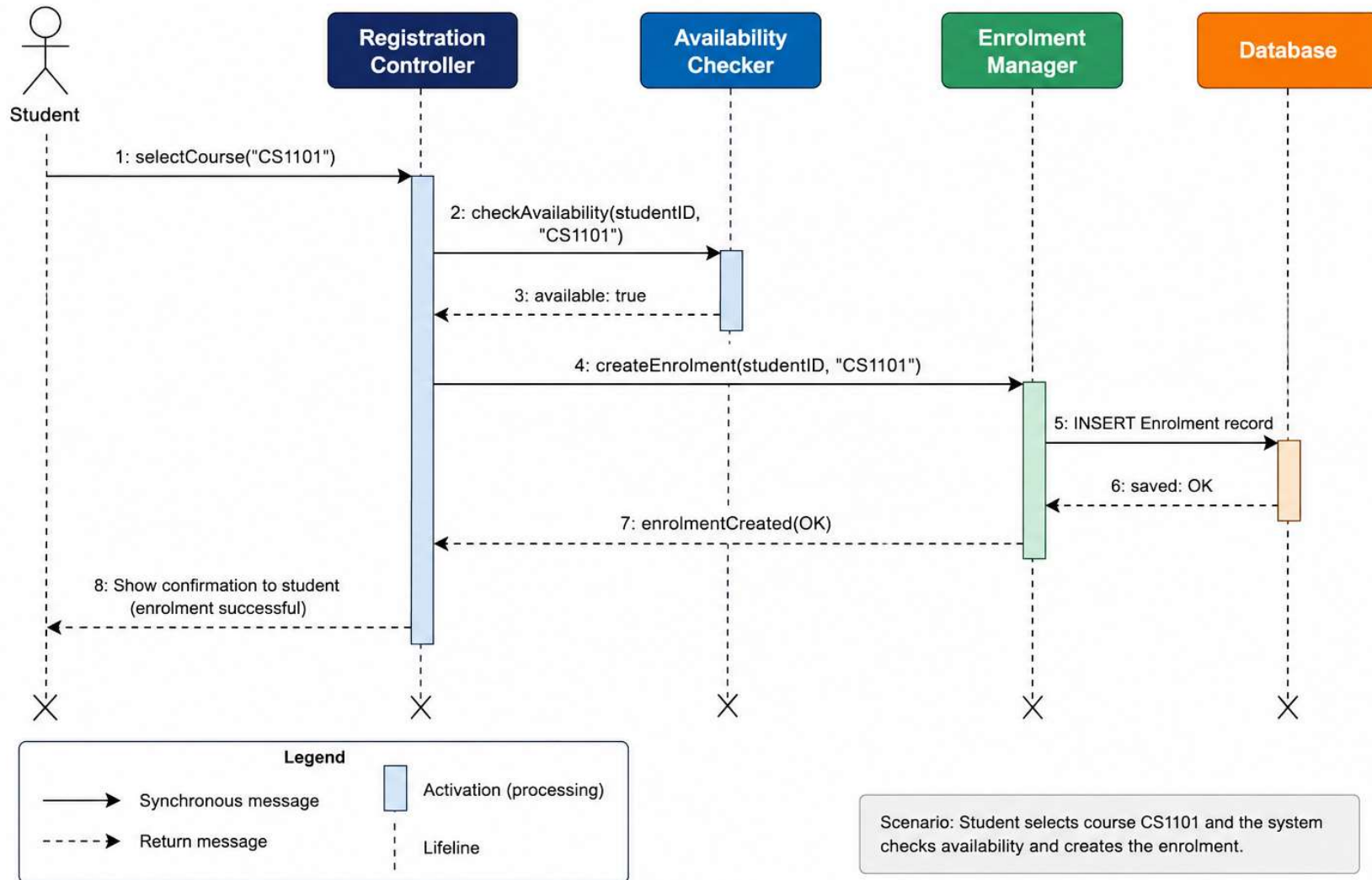
Shows who communicates with whom and in what order

Answers: What is the step-by-step message flow between components for one specific action?

Example: Student registers for CS1101



UML 3: Sequence Diagram – Student registers for CS1101

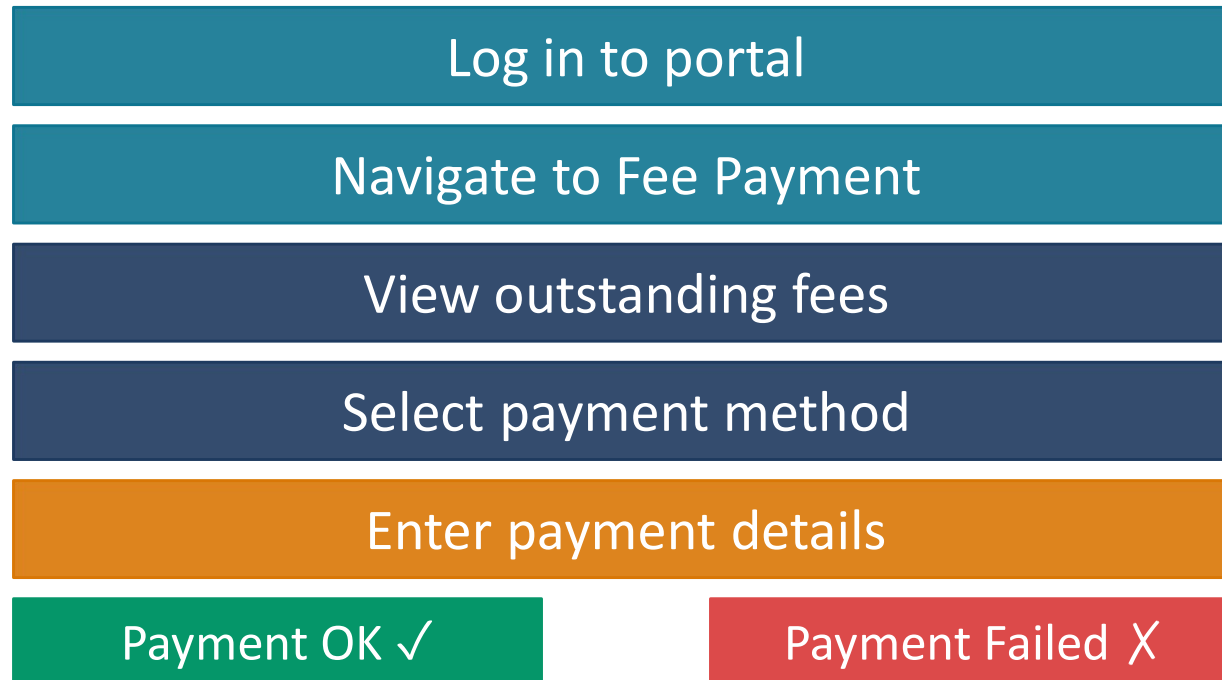


UML 4: Activity Diagram

Shows the flow of actions in a process — decisions and parallel steps included

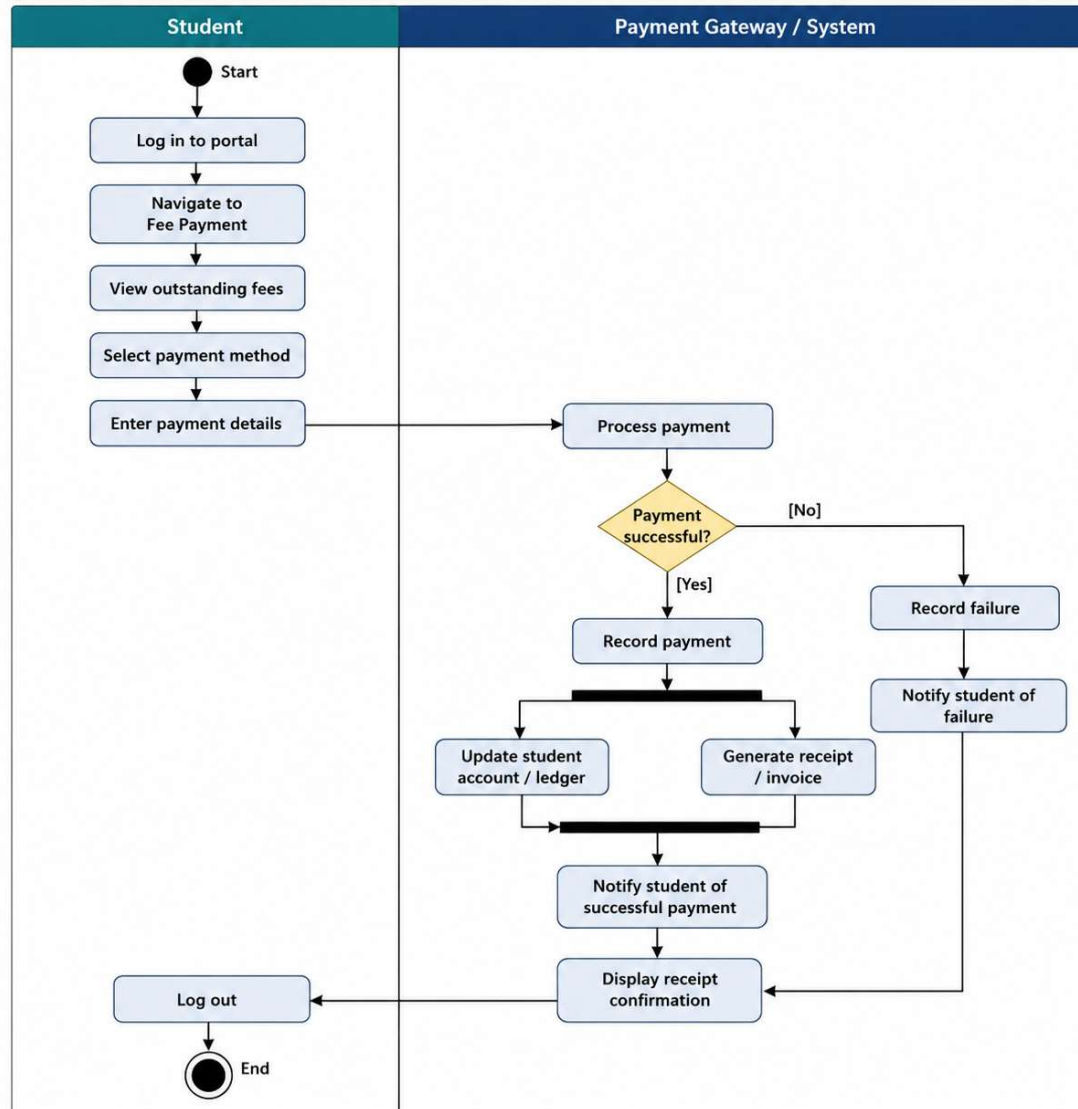
Answers: What are the steps in a process? Where do decisions happen? Can some steps run in parallel?

Example: Student pays fees in the portal



Key notation: rounded rectangle = activity · diamond = decision · filled circle = start · arrows = flow direction

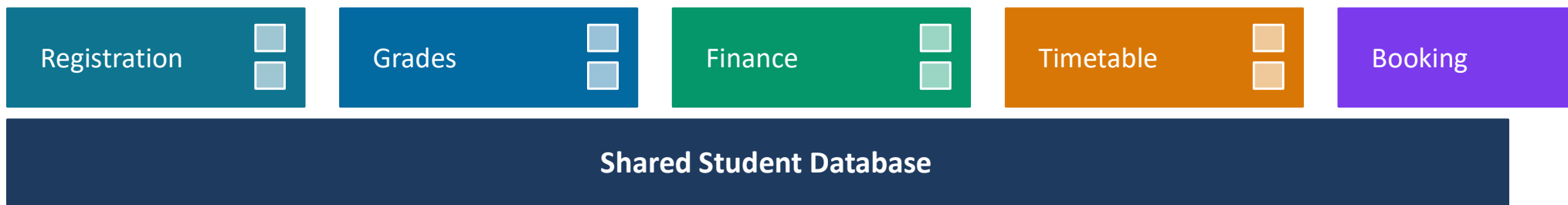
Student Pays Fees in the Portal – Activity Diagram



UML 5 and 6: Component and Deployment Diagrams

What modules exist — and where do they physically run?

Component Diagram — the software modules and their dependencies

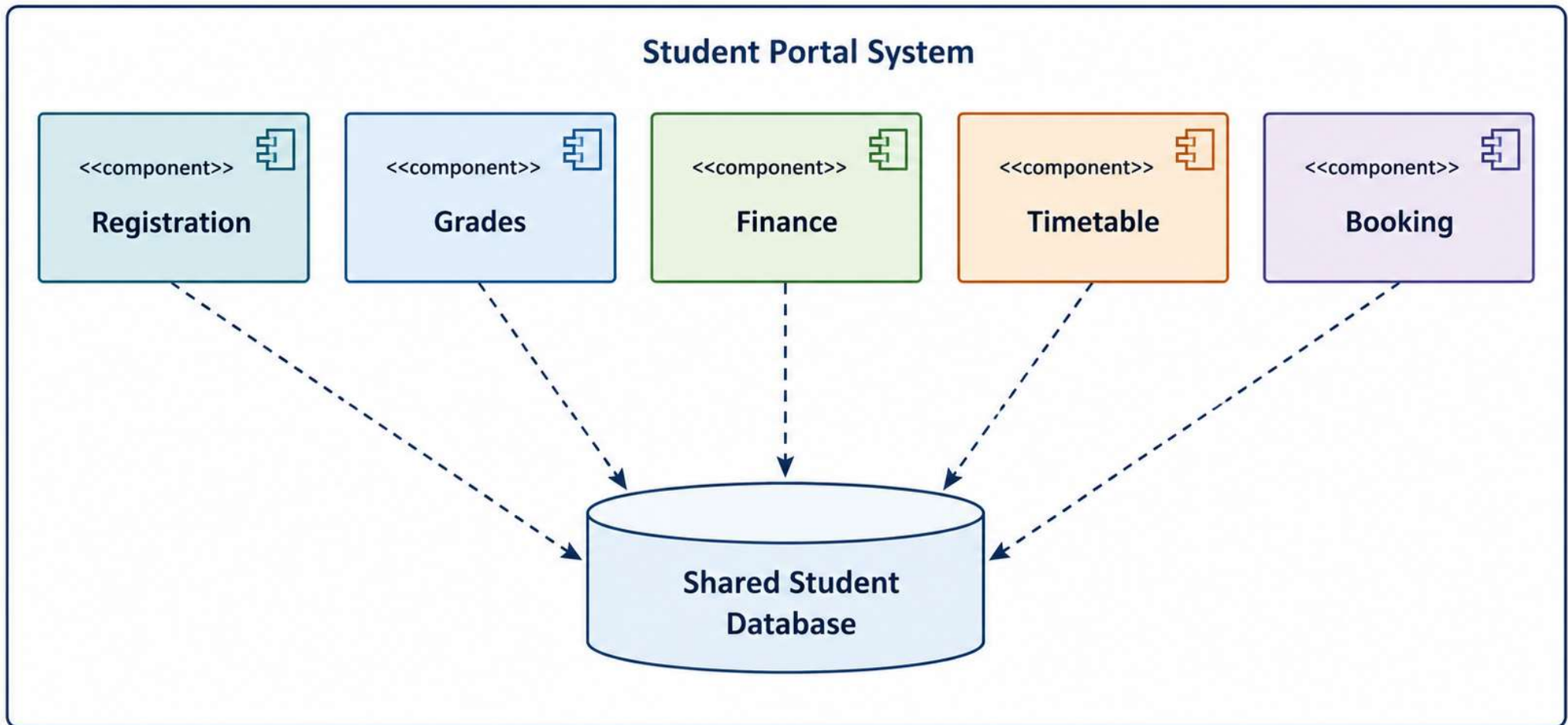


Deployment Diagram — where the software physically runs

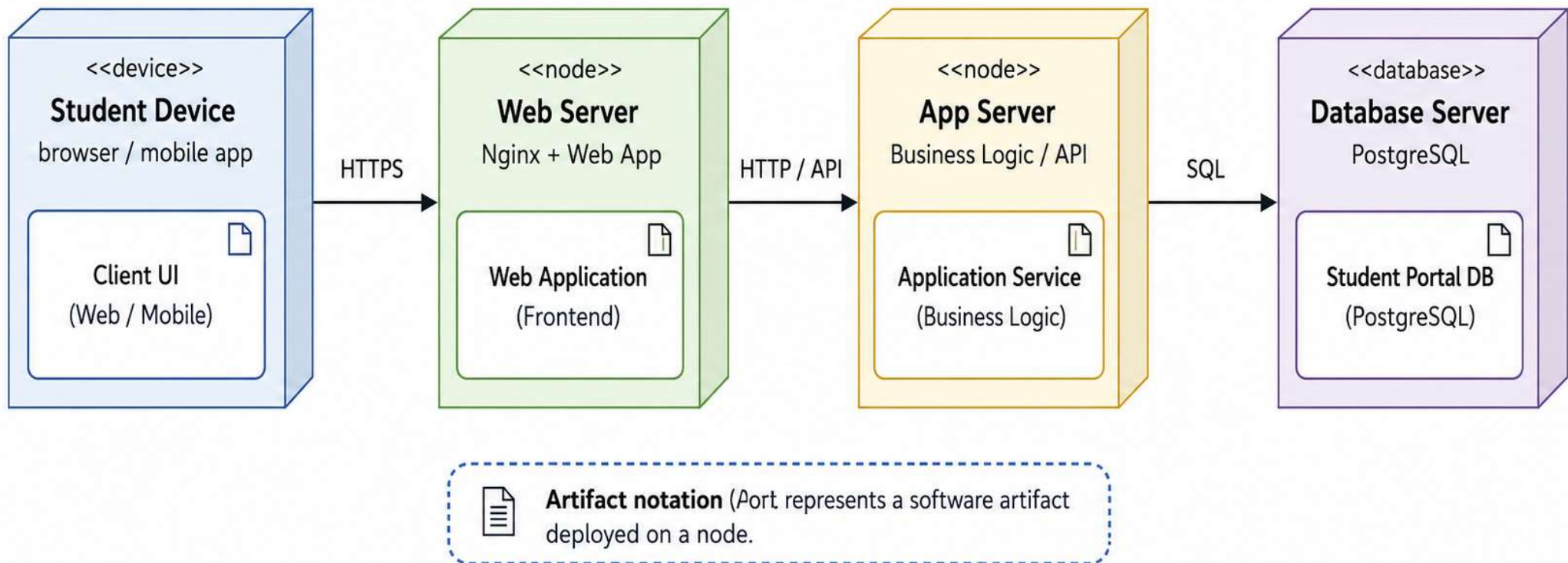


Running example: UCSC Student Portal

UML Component Diagram – Student Portal



UML Deployment Diagram – Student Portal System



Deployment diagram shows where software artifacts are physically deployed.

Choosing the Right UML Diagram

Match the diagram to the question you are trying to answer

Question you are asking	Diagram	Why this one
What can each user do with the system?	Use Case	Shows actors and their interactions — external view
What are the main entities and their data?	Class	Shows the static structure — things and relationships
What messages flow between objects, in what order?	Sequence	Shows time-ordered message passing for one scenario
What are the steps in this business process?	Activity	Shows action flow including decisions and branches
What software modules make up the system?	Component	Shows modules, their interfaces, and dependencies
What hardware runs the system?	Deployment	Shows servers, devices, and where software components live

Section 5.6

Design in Both Cultures

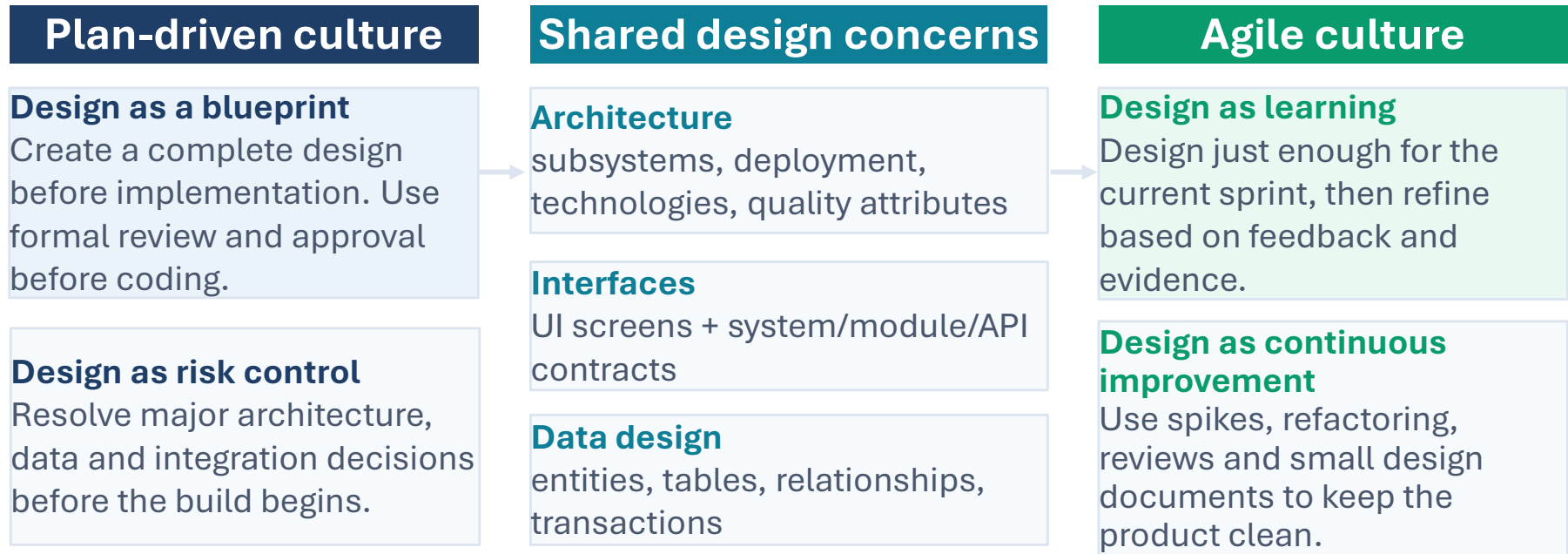
How design is practised in plan-driven and agile contexts

Design in Plan-Driven and Agile Development

Same design goal — different timing, detail and control

The important message :

Agile does not remove design. It changes how design is discovered, documented, reviewed and improved.



Compare plan-driven and agile by timing, documentation depth, feedback and change control — not by “design” versus “no design”.

Plan-Driven Design: Complete Blueprint Before Build

Useful when requirements are stable, risk is high, or formal approval is required

Design sequence

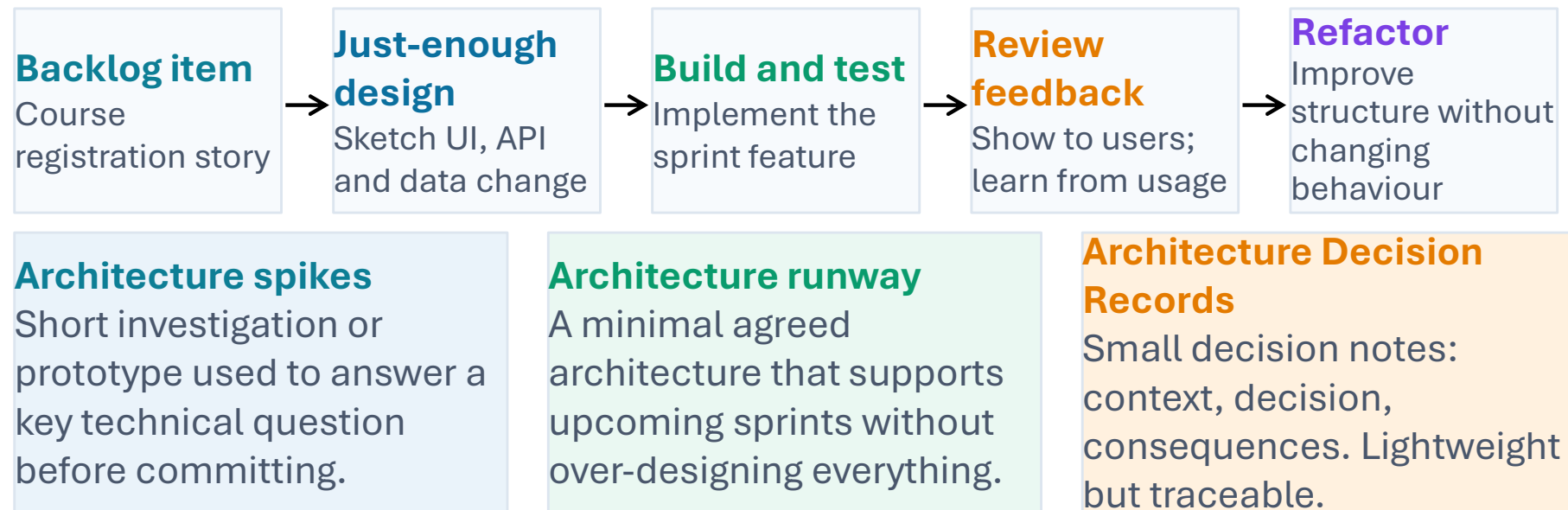
- 1 Requirements specification**
Complete SRS signed off by stakeholders
- 2 High-Level Design (HLD)**
Architecture, major subsystems, patterns, deployment decisions
- 3 Detailed Design (LLD)**
Component internals, interfaces, UI screens, data schema
- 4 Design review and sign-off**
Stakeholders approve the design before coding starts
- 5 Implementation**
Developers build according to the approved design

In plan-driven development, design is treated as a formally approved contract before implementation begins.

Agile Design: Enough Design, Then Continuous Improvement

Architecture remains important — the detail emerges sprint by sprint

Agile design loop



Agile design evidence is usually lighter: user stories, acceptance criteria, sketches, tests, small diagrams, ADRs and code review notes.

Agile teams reduce waste by delaying detailed decisions until they have enough knowledge — but they still protect the architecture.

Level 3 in Agile or Plan-Driven: UI + System Interfaces

Interface design has two audiences: users and software components

3A. User Interface Design

Focus

How external users interact with the system through screens, forms, navigation and feedback.

Typical evidence

Wireframes, screen catalogue, field rules, messages, accessibility and responsive behaviour.

Student Portal example

Course Registration Screen: search course, select semester, register, show success or error.

3B. System Interface Design

Focus

How modules, services and databases communicate through contracts and messages.

Typical evidence

API signatures, request/response formats, preconditions, postconditions, errors and security rules.

Student Portal example

```
checkAvailability(studentID, courseCode)
→ { available, reason }
```

Recommended teaching change: split Level 3 into User Interface Design and System Interface Design so students do not confuse screens with APIs.

Level 4: Data Design Is a Separate Critical Design Area

Data design defines how business information is organised, protected and changed

Data design decisions

Conceptual model

Main entities and relationships:
Student, Course,
Enrolment, Result,
Payment.

Logical schema

Tables, columns, data
types, primary keys
and foreign keys.

Rules and constraints

Uniqueness,
mandatory fields,
allowed status values,
referential integrity.

Transactions and performance

Atomic registration,
locking, indexes and
common query paths.

Example table: Enrolment

enrolmentID		studentID		courseCode		semester		status		registrationDate
-------------	--	-----------	--	------------	--	----------	--	--------	--	------------------

Design rule

A student can have many enrolments. One enrolment must reference exactly one student and one course.

Transaction rule

When a student registers, availability check and enrolment insert should be completed safely as one controlled operation.

Poor data design can break registration, reporting, security and future integration.

Plan-Driven vs Agile Design — Side by Side

A clearer comparison for students

Aspect	Plan-Driven Design	Agile Design
Timing	Most design is completed before coding.	Design is done just in time for the current sprint.
Level of detail	Full HLD, LLD, UI, API and database detail early.	High-level direction early; details emerge incrementally.
Documentation	Formal SDS/HLD/LLD documents and approval.	Lightweight evidence: sketches, ADRs, tests, key diagrams.
Change control	Formal change request; changes can be costly.	Change welcomed; design adapts through refactoring.
Risk handling	Predict risks early and plan mitigation.	Discover risks early using spikes and short feedback loops.

Most real projects blend both: agree on important architecture early, then refine UI, interfaces and data details sprint by sprint.

Student Portal Example: Same Feature, Two Design Cultures

Course registration can be designed using either approach

Plan-driven delivery

1 Before coding

Complete UI screens, API contracts, database schema and sequence diagrams.

2 Approval

Review and sign off SDD before implementation.

3 Coding

Build exactly according to the approved design.

Agile delivery

1 Sprint 1

Search and view available courses; lightweight UI and API design.

2 Sprint 2

Add register action; refine Enrolment table and validation rules.

3 Sprint 3

Add conflict checking, error messages, logging and refactoring.

both cultures design the same things — architecture, UI, interfaces and data — but with different timing and documentation weight.

Design is a decision-making activity. The process culture decides when, how much and how formally those decisions are recorded.

Section 5.7

Design Specification

Recording design decisions so developers can build — and maintainers can understand

What is a Design Specification?

The document that guides developers from design to code

Purpose

A design specification records the design decisions made so that developers, testers, and maintainers can understand and implement the system correctly. It is the bridge between thinking about the design and writing the code.

Plan-driven context — three levels of specification:

High-Level Design (HLD)

The architectural view — subsystems, patterns, and how they connect. Audience: architects, senior developers, project managers.

Low-Level Design (LLD)

Component internals — class structures, algorithms, interface contracts. Audience: developers who will write the code.

Software Design Document (SDD)

The complete design specification: HLD + LLD + UML diagrams + design rationale. Audience: the whole project team.

Design Specification in Agile

Lightweight models — enough to communicate, not to create bureaucracy

In agile, the goal is working software — not comprehensive documentation. Design is specified in the lightest form that still communicates clearly.

User stories with acceptance criteria

The requirement and the design intent in one place. The acceptance criteria define testable behaviour that the implementation must satisfy.

Quick whiteboard diagrams

A rough class or sequence diagram sketched in a sprint planning session. Photographed and stored — does not need to be perfectly formatted.

Architecture decision records (ADRs)

Short written records of major architectural decisions: What was decided? Why? What alternatives were considered? This preserves the reasoning for future teams.

Definition of Done

D97706

"Documentation is not the goal — shared understanding is. Use the minimum documentation needed to achieve it." — Agile principle 6

Section 5.X

Major Approaches in Software Design

There is no single “best” approach. In real projects, designers combine several approaches depending on the problem.

Major Approaches in Software Design

A design approach is a way of thinking about how to organise a system

There is no single “best” approach. In real projects, designers combine several approaches depending on the problem.

1

Flow of steps
Procedural /
workflow design

2

Things in the
system
Object-oriented
design

3

Replaceable
modules
Component-
based design

4

Layers and
deployment
Architectural
design styles

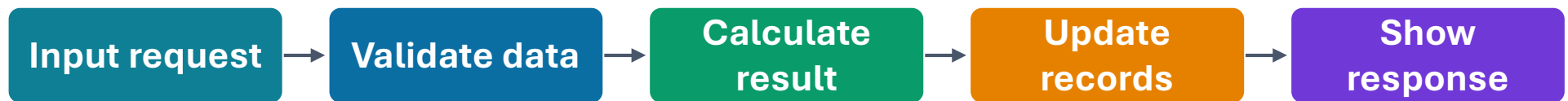
5

Data and events
Data-centred /
event-driven
design

For the Student Portal, we use multiple approaches: use cases for user goals, classes for data and behaviour, components for subsystems, interfaces for communication, and database design for persistent data.

1. Procedural Design

Design the system as a sequence of procedures or steps



Main design question

What steps must happen, and in what order?

Good for

Simple workflows, algorithms, scripts, batch processing, and step-by-step business processes.

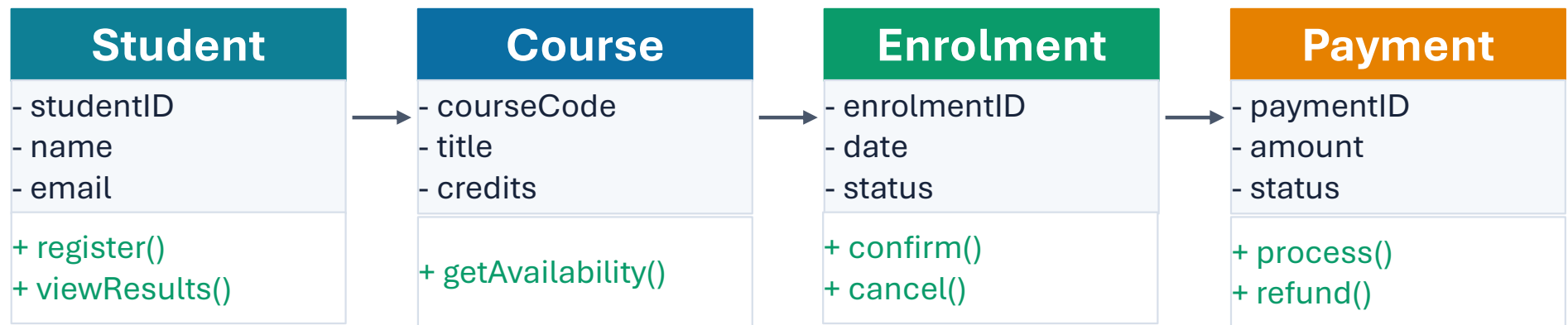
Limitation

If the system grows, many procedures may share global data and become hard to maintain.

**Student Portal example: “Pay fees” can be designed first as a procedure:
log in → view fees → select method → pay → record payment → show receipt.**

2. Object-Oriented Design

Design the system around objects that combine data and behaviour



Main design question

What are the important things in the system, what data do they hold, and what behaviour belongs to each?

Why useful

Encapsulation keeps related data and operations together. This improves maintainability and reuse.

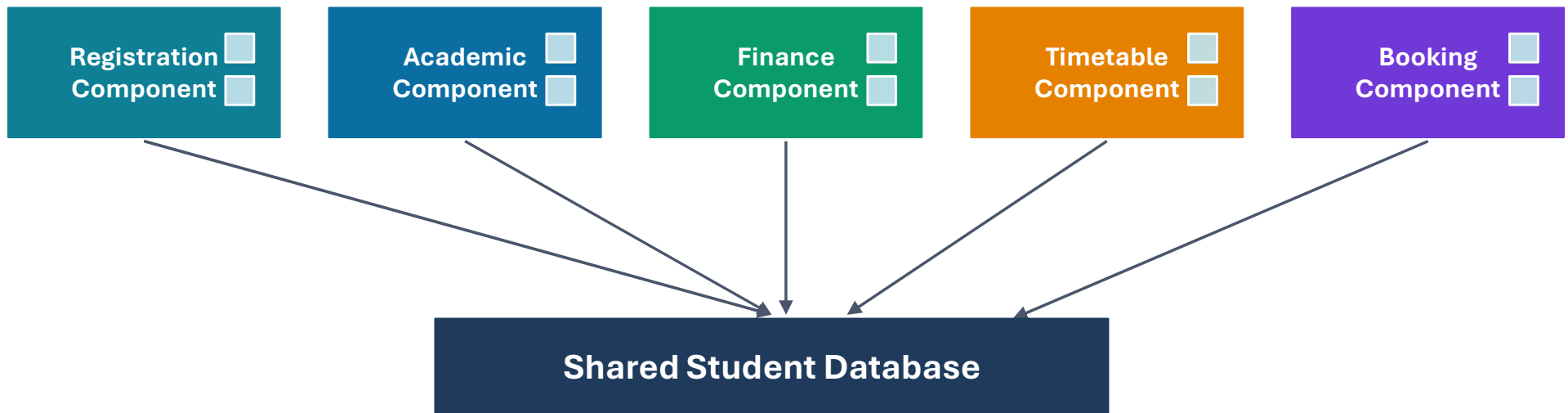
Watch out

Do not create classes only because nouns appear in requirements. Classes must have clear responsibility.

Related UML diagram: Class Diagram

3. Component-Based Design

Design the system as independent modules with clear interfaces



Main design question

How can the system be divided into replaceable, testable, independently understandable modules?

Key design output

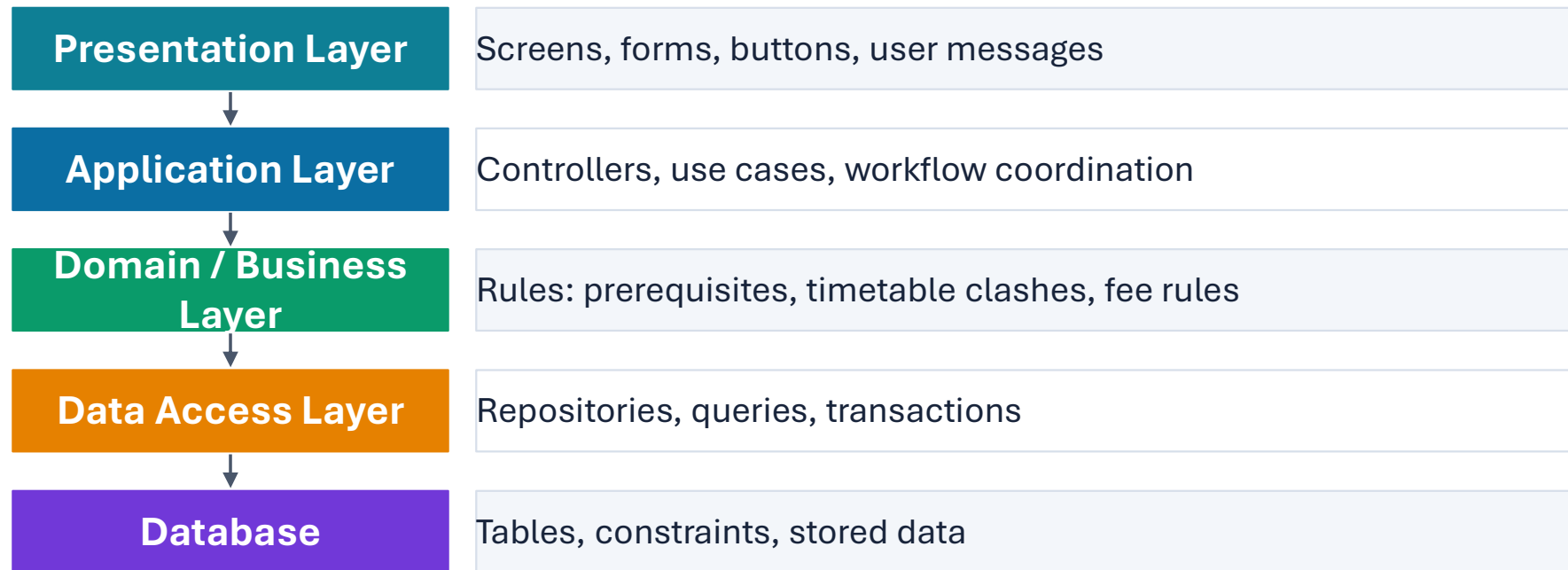
Component responsibilities, required/provided interfaces, dependencies, and ownership.

Student Portal example

Registration uses CourseAvailabilityChecker and EnrolmentManager through defined interfaces.

4. Layered / Tiered Design

Separate responsibilities so each layer has a clear job



Why this matters *UI changes should not require database redesign. Database changes should not directly affect every screen. Layers control dependency direction.*

5. Service-Oriented / API-Based Design

Design separate services that communicate through defined APIs



API contracts

```
POST /registrations
GET /courses/{courseCode}/availability
POST /payments
POST /notifications
```

When useful

Different teams own different areas; services must be changed and deployed separately.

Important warning

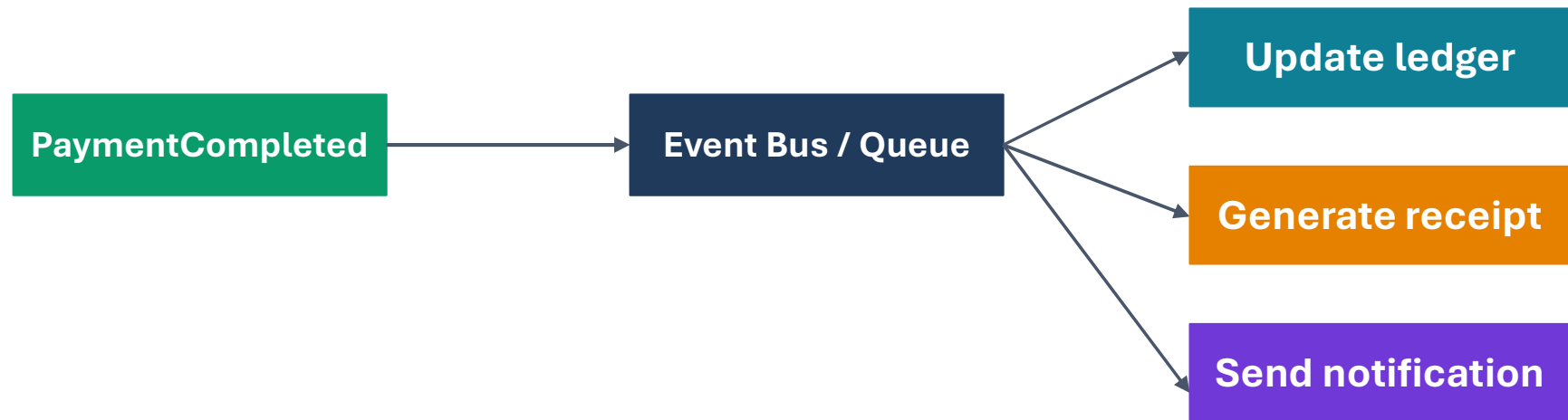
More services add communication, testing, monitoring, and deployment complexity.

Microservices

A microservice is a smaller, independently deployable service, not just a small class.

6. Event-Driven Design

Components communicate by publishing and reacting to events



Main design question

What important events happen in the system, and which components should react?

Good for

Notifications, audit logs, background processing, integration with external systems.

Watch out

Event flows can become hard to trace unless events, ownership, and failure handling are documented.

7. Functional and Data-Centred Design

Useful supporting ways of thinking inside modern systems

Functional design idea

Design logic as small, predictable functions.

Example:

`validatePrerequisites(student, course) → true/false`

`calculateGPA(results) → GPA`

`calculateOutstandingFees(student) → amount`

Data-centred design idea

Design around the data model, database rules, and data ownership.

Example:

Student, Course, Enrolment, Payment, Result

Questions:

Who owns this data?

What constraints protect it?

What transaction must be atomic?

Functional design is usually not the whole architecture of a Student Portal, but it is very useful for validation rules, calculations, and transformations.

How to Choose a Design Approach

Choose based on the design problem, not based on fashion

If the question is...	Use this approach	Useful diagram / document
What steps happen?	Procedural / workflow	Activity diagram, sequence diagram
What things and relationships exist?	Object-oriented	Class diagram
How do we split the system?	Component-based	Component diagram, HLD/LLD
Where should responsibilities live?	Layered / tiered	Architecture diagram
How do modules communicate?	API / system interface	Interface specification
How is persistent data organised?	Data-centred	ERD, database design document

Best message: choose the design lens that answers the current design question.

Student Portal: A Blended Design

The same case study uses several approaches together

Use cases

Who uses the system and what goals they have

User interface design

What screens, forms, validation, and feedback users see

Class design

What entities, data, behaviour, and relationships exist

Component design

What modules exist and what each module is responsible for

System interface design

How modules call each other through APIs or function contracts

Data design

What tables, keys, constraints, transactions, and indexes protect data

One design specification combines these parts into one consistent blueprint.

Section 5.8

AI and Tools in Software Design

How modern tools — and AI — are changing the practice of design

Design Tools — What Engineers Use Today

Before AI: engineers designed on whiteboards, in drawing tools, and in IDEs. Today: AI is embedded in most of these tools, offering suggestions, generating code, and reviewing designs.

Diagramming tools

Draw.io (free), Lucidchart, Microsoft Visio — for creating UML diagrams, architecture sketches, and data flow diagrams

IDE with AI assistance

Visual Studio Code, IntelliJ IDEA — modern IDEs include GitHub Copilot and similar AI code completion built in

Architecture and modelling

Enterprise Architect, StarUML — dedicated tools for large-scale UML modelling and architecture documentation

Collaborative design

Miro, Figma — virtual whiteboards for remote teams to design together in real time

AI design assistants

ChatGPT, Claude, GitHub Copilot — generate design suggestions, review architecture, write class structures, explain patterns

AI in Software Design — What It Can Do

The landscape is changing rapidly — knowing this now gives you a head start

The shift

AI tools are being integrated into every stage of the design process — from generating component structures to reviewing architecture decisions. The engineer's role is shifting to directing, reviewing, and validating AI-generated design work.

Where AI is helpful in design today:

Code and class generation

GitHub Copilot, Cursor — AI suggests class structures, method signatures, boilerplate, and patterns as you type in the IDE

Architecture review

Describe your architecture to ChatGPT or Claude — AI identifies potential problems, suggests improvements, and explains trade-offs

UML diagram generation

Tools like PlantUML with AI describe a system in text; AI produces a UML diagram. From text description to diagram in seconds.

Design pattern suggestions

Describe a design problem — AI suggests which design pattern fits and shows example code for your context

Refactoring assistance

Paste code and ask AI to identify cohesion and coupling problems and suggest improvements — a code review partner available instantly

What AI Cannot Do in Design

Professional judgement, responsibility, and ethical thinking remain human

AI is a powerful tool — not a replacement for engineering knowledge and professional responsibility.

⚠️ AI can generate designs — but cannot evaluate business context

AI does not know your users, your organisation, your constraints, or your business goals. The engineer must provide that context and judge whether the AI's suggestion fits.

⚠️ AI may suggest patterns that are over-engineered

AI often suggests complex solutions. The principle of simplicity still applies — the engineer must judge whether the suggested design is the SIMPLEST that meets the requirements.

⚠️ AI output must be reviewed and understood

Never paste AI-generated design into a system you do not understand. You are responsible for the design — not the AI that suggested it.

⚠️ AI learns from existing code — it can propagate bad habits

AI is trained on vast amounts of existing code, including poorly designed code. It can confidently suggest designs that violate cohesion, coupling, or information hiding principles.

⚠️ Professional accountability stays with the engineer

If an AI-suggested architecture leads to a system failure, the engineer who accepted it is responsible. Professional accountability does not transfer to the tool.

Common Design Pitfalls — What Goes Wrong

Knowing what goes wrong helps you avoid it in your own projects:

⚠ Skipping the architecture

Starting to code before deciding the structure. Discovered late — structural changes are expensive and disruptive.

⚠ Over-engineering

Designing for every possible future scenario. Complexity you add now must be maintained forever. Design for what is needed — not for every what-if.

⚠ God class / God function

One class or function that does everything. Zero cohesion, maximum coupling. The first sign of design breakdown.

⚠ Tight coupling everywhere

Every component knows the internals of every other component. A change anywhere breaks everything. The second sign of breakdown.

⚠ Skipping design documentation

'I'll just write the code' — nobody documents the design decisions. Three months later, nobody knows why anything was built the way it was.

⚠ No design review

Design is never reviewed before implementation begins. Problems that would take 10 minutes to fix at the design stage take weeks to fix in code.

Section 5 — What We Have Covered

Check your understanding against the learning outcomes

LO1

Software design transforms requirements into a blueprint. It defines what the parts are, how they connect, and what each part does — before any code is written.

LO2

The design journey goes top-down: architectural design → component design → interface design → data design → algorithmic detail. Architecture always comes first.

LO3

Five core principles: Abstraction (hide detail), Modularity (divide into pieces), Cohesion (one thing well), Low Coupling (minimise dependencies), Information Hiding (hide internals).

LO4

Architectural patterns: Layered (horizontal layers), Client-Server (client requests / server responds), MVC (model / view / controller), Repository (shared central data store).

LO5

System modelling uses four perspectives (external, interaction, structural, behavioural). Six key UML diagrams: Use Case, Class, Sequence, Activity, Component, Deployment.

LO6

Plan-driven: Big Design Upfront — complete SDD before coding. Agile: emergent design — architecture spikes, just-in-time design, refactoring after each sprint.

LO7

AI tools assist with class generation, architecture review, UML diagram generation, pattern suggestions, and refactoring. Engineers must review, understand, and own every design decision.

References and Further Reading

Main Textbooks

Software Engineering

Ian Sommerville, 10th edition, Addison-Wesley, 2015

Chapters 6 (Architectural Design) and 7 (Design and Implementation) — primary reading

Software Engineering: A Practitioner's Approach

Roger S. Pressman, 9th edition, McGraw-Hill, 2020

Chapters 9–12: Design concepts, architectural design, component design, user interface design

Clean Architecture

Robert C. Martin, Prentice Hall, 2017

An accessible, practical guide to design principles and architectural patterns — highly recommended

Online Resources

- [draw.io](#) — free, browser-based UML and architecture diagram tool (no installation required)
- [refactoring.guru](#) — free guide to design patterns with examples in multiple programming languages
- [github.com/features/copilot](#) — GitHub Copilot overview and how to start using it
- [c4model.com](#) — the C4 model: a modern, lightweight approach to documenting software architecture

Coming Up Next

Section 6: System Implementation

From design to code · Programming languages · Code quality · Testing strategies · Version control

Tutorial this week:

Draw a class diagram for the Student Portal · Sketch the sequence for 'Student views exam results' · Identify one architectural pattern in a system you use daily

Introduction to Software Engineering · Prof. K.
P. Hewagamage